

Import & Nettoyage

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: # =====
# 1. IMPORT DES DATASETS ESSENTIELS
# =====

print("Chargement des données...")

# WDI - World Development Indicators
wdi = pd.read_csv('/Users/cordelia/Documents/Projet 11/New_notebook/WDI_D

# WGI - World Governance Indicators
wgi = pd.read_csv('/Users/cordelia/Documents/Projet 11/New_notebook/WGI_D

# CEPII - Distances géographiques
dist_cepii = pd.read_excel('/Users/cordelia/Documents/Projet 11/New_noteb
geo_cepii = pd.read_excel('/Users/cordelia/Documents/Projet 11/New_notebo

# FAO - Food Balance (consommation viande)
food_balance = pd.read_csv('/Users/cordelia/Documents/Projet 11/New_noteb

# FAO - Trade Detailed (imports volaille)
trade_detailed = pd.read_csv('/Users/cordelia/Documents/Projet 11/New_not

# FAO - Emissions Livestock (émissions volaille)
emissions_livestock = pd.read_csv('/Users/cordelia/Documents/Projet 11/Ne

print("✓ Toutes les données chargées")
```

Chargement des données...

✓ Toutes les données chargées

```
In [3]: # =====
# 2. VÉRIFICATION RAPIDE
# =====

print("\n=== SHAPES ===")
print(f"WDI: {wdi.shape}")
print(f"WGI: {wgi.shape}")
print(f"Distance CEPII: {dist_cepii.shape}")
print(f"Geo CEPII: {geo_cepii.shape}")
print(f"Food Balance: {food_balance.shape}")
print(f"Trade Detailed: {trade_detailed.shape}")
print(f"Emissions Livestock: {emissions_livestock.shape}")

print("\n✓ Notebook prêt pour le nettoyage")
```

```
=== SHAPES ===  
WDI: (5591, 10)  
WGI: (7709, 10)  
Distance CEPII: (50176, 14)  
Geo CEPII: (238, 34)  
Food Balance: (4820497, 14)  
Trade Detailed: (52410630, 16)  
Emissions Livestock: (6650421, 16)
```

✓ Notebook prêt pour le nettoyage

Nettoyage WDI

```
In [4]: # 1. Afficher les indicateurs disponibles  
print("\n--- Indicateurs disponibles dans WDI ---")  
print(wdi['Series Name'].unique())
```

--- Indicateurs disponibles dans WDI ---

```
<StringArray>
[
    'GDP per capita (current US
$)',
    'GDP growth (annual
%)',
    'GNI per capita, Atlas method (current US
$)',
    'Trade (% of GD
P)',
    'Foreign direct investment, net inflows (BoP, current US
$)',
    'Inflation, consumer prices (annual
%)',
    'Exports of goods and services (% of GD
P)',
    'Population, tot
al',
    'Urban population (% of total populatio
n)',
    'Life expectancy at birth, total (year
s)',
    'Literacy rate, adult total (% of people ages 15 and abov
e)',
    'Gini ind
ex',
    'Population growth (annual
%)',
    'Access to electricity (% of populatio
n)',
    'Individuals using the Internet (% of populatio
n)',
    'Logistics performance index: Overall (1=low to 5=hig
h)',
    'Carbon dioxide (CO2) emissions excluding LULUCF per capita (t CO2e/capit
a)',
    'Agricultural land (% of land are
a)',
    'Electric power consumption (kWh per capit
a)',
    'Tariff rate, applied, weighted mean, all products
(%)',
    'Current health expenditure per capita (current US
$)',
    nan]
Length: 22, dtype: str
```

```
In [5]: print("\n STRUCTURE WDI")
print("\n*80)
print(f"Shape: {wdi.shape}")
print(f"\nColonnes:")
print(wdi.columns.tolist())
print(f"\nPremières lignes:")
print(wdi.head(10))
print(f"\nInfo sur les types:")
print(wdi.info())
```

STRUCTURE WDI

```
=====
```

```
=====
```

```
Shape: (5591, 10)
```

```
Colonnes:
```

```
['Country Name', 'Country Code', 'Series Name', 'Series Code', '2019 [YR2019]', '2020 [YR2020]', '2021 [YR2021]', '2022 [YR2022]', '2023 [YR2023]', '2024 [YR2024]']
```

```
Premières lignes:
```

```
Country Name Country Code \
0 Afghanistan AFG
1 Afghanistan AFG
2 Afghanistan AFG
3 Afghanistan AFG
4 Afghanistan AFG
5 Afghanistan AFG
6 Afghanistan AFG
7 Afghanistan AFG
8 Afghanistan AFG
9 Afghanistan AFG
```

```
Series Name Series Code \
0 GDP per capita (current US$) NY.GDP.PCAP.CD
1 GDP growth (annual %) NY.GDP.MKTP.KD.ZG
2 GNI per capita, Atlas method (current US$) NY.GNP.PCAP.CD
3 Trade (% of GDP) NE.TRD.GNFS.ZS
4 Foreign direct investment, net inflows (BoP, c... BX.KLT.DINV.CD.WD
5 Inflation, consumer prices (annual %) FP.CPI.TOTL.ZG
6 Exports of goods and services (% of GDP) NE.EXP.GNFS.ZS
7 Population, total SP.POP.TOTL
8 Urban population (% of total population) SP.URB.TOTL.IN.ZS
9 Life expectancy at birth, total (years) SP.DYN.LE00.IN
```

```
2019 [YR2019] 2020 [YR2020] 2021 [YR2021] 2022 [YR2022]
2] \
0 496.6025042585 510.787063366811 356.496214115892 357.2611527981
44
1 3.91160341625552 -2.35110067203466 -20.7388393676343 -6.240171992402
69
2 520 490 380 3
70
3 .. 46.7098945514103 51.4117163747271 72.88546960865
36
4 23404553.647249 12970147.9832 20600980
0
5 2.30237251516834 5.60188791482224 5.13320340824963 13.71210237200
65
6 .. 10.4208171386207 14.342152797537 18.38004240163
75
7 37856121 39068979 40000412 405788
42
8 25.1437258839803 25.2622103446137 25.3478620272481 25.39392505809
09
9 62.941 61.454 60.417 65.6
17

2023 [YR2023] 2024 [YR2024]
0 413.757894705303 ..
```

```

1  2.26694373649188    ..
2                370    ..
3  67.5846651516385    ..
4                0      0
5 -4.64470870797775  -6.60118564073726
6  16.8527881206002    ..
7        41454761      42647492
8  25.4730529631167  25.7007352636188
9        66.035        ..

```

Info sur les types:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 5591 entries, 0 to 5590

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	Country Name	5588 non-null	str
1	Country Code	5586 non-null	str
2	Series Name	5586 non-null	str
3	Series Code	5586 non-null	str
4	2019 [YR2019]	5586 non-null	str
5	2020 [YR2020]	5586 non-null	str
6	2021 [YR2021]	5586 non-null	str
7	2022 [YR2022]	5586 non-null	str
8	2023 [YR2023]	5586 non-null	str
9	2024 [YR2024]	5586 non-null	str

dtypes: str(10)

memory usage: 436.9 KB

None

```

In [6]: #Filtrer les 8 indicateurs
indicateurs_a_garder = [
    'GDP per capita (current US$)',
    'GDP growth (annual %)',
    'Inflation, consumer prices (annual %)',
    'Population, total',
    'Urban population (% of total population)',
    'Logistics performance index: Overall (1=low to 5=high)',
    'Carbon dioxide (CO2) emissions excluding LULUCF per capita (t CO2e/c',
    'Tariff rate, applied, weighted mean, all products (%)'
]

wdi_clean = wdi[wdi['Series Name'].isin(indicateurs_a_garder)].copy()
print(f"✓ Après filtrage: {wdi_clean.shape}")
print(f"Indicateurs gardés: {wdi_clean['Series Name'].nunique()}/8")

```

✓ Après filtrage: (2128, 10)

Indicateurs gardés: 8/8

```

In [7]: #Convertir les ".." en NaN et les colonnes en float
year_cols = ['2019 [YR2019]', '2020 [YR2020]', '2021 [YR2021]',
             '2022 [YR2022]', '2023 [YR2023]', '2024 [YR2024]']

for col in year_cols:
    # Remplacer ".." par NaN
    wdi_clean[col] = wdi_clean[col].replace('..', np.nan)
    # Convertir en float
    wdi_clean[col] = pd.to_numeric(wdi_clean[col], errors='coerce')

print("\n✓ Conversion en numérique terminée")

```

✅ Conversion en numérique terminée

```
In [8]: #Vérifier le taux de NaN par année
print("\n📊 Taux de NaN par année:")
for col in year_cols:
    nan_pct = (wdi_clean[col].isna().sum() / len(wdi_clean)) * 100
    print(f"{col}: {nan_pct:.1f}% NaN")
```

📊 Taux de NaN par année:
 2019 [YR2019]: 21.7% NaN
 2020 [YR2020]: 21.0% NaN
 2021 [YR2021]: 21.0% NaN
 2022 [YR2022]: 13.0% NaN
 2023 [YR2023]: 29.2% NaN
 2024 [YR2024]: 30.4% NaN

```
In [9]: #Pivoter en format WIDE pour l'année 2023
wdi_2023 = wdi_clean[['Country Name', 'Country Code', 'Series Code', '2023 [YR2023]']]

wdi_wide = wdi_2023.pivot(
    index=['Country Name', 'Country Code'],
    columns='Series Code',
    values='2023 [YR2023]'
).reset_index()

print(f"\n✅ Format WIDE créé: {wdi_wide.shape}")
print(f"Pays: {len(wdi_wide)}")
print(f"Variables: {len(wdi_wide.columns) - 2}") # -2 pour Country Name
```

✅ Format WIDE créé: (266, 10)
 Pays: 266
 Variables: 8

```
In [10]: #Renommer les colonnes avec des noms courts
rename_dict = {
    'NY.GDP.PCAP.CD': 'gdp_per_capita',
    'NY.GDP.MKTP.KD.ZG': 'gdp_growth',
    'FP.CPI.TOTL.ZG': 'inflation',
    'SP.POP.TOTL': 'population',
    'SP.URB.TOTL.IN.ZS': 'urban_pop_pct',
    'LP.LPI.OVRL.XQ': 'logistics_index',
    'EN.GHG.CO2.PC.CE.AR5': 'co2_per_capita',
    'TM.TAX.MRCH.WM.AR.ZS': 'tariff_rate'
}

wdi_wide = wdi_wide.rename(columns=rename_dict)

print("\n✅ Colonnes renommées")
print("\nColonnes finales:")
print(wdi_wide.columns.tolist())
```

✅ Colonnes renommées

Colonnes finales:
 ['Country Name', 'Country Code', 'co2_per_capita', 'inflation', 'logistics_index', 'gdp_growth', 'gdp_per_capita', 'population', 'urban_pop_pct', 'tariff_rate']

```
In [11]: #Statistiques de complétude
print("\n" + "="*80)
print("📊 ANALYSE DE COMPLÉTUDE (2023)")
```

```
print("="*80)

# Pays avec données complètes
pays_complets = wdi_wide.dropna()
print(f"Pays avec TOUTES les variables: {len(pays_complets)}")

# NaN par variable
print("\nNaN par variable:")
for col in wdi_wide.columns[2:]: # Skip Country Name et Code
    nan_count = wdi_wide[col].isna().sum()
    nan_pct = (nan_count / len(wdi_wide)) * 100
    print(f" {col:20s}: {nan_count:3d} NaN ({nan_pct:5.1f}%)")

# Afficher les premières lignes
print("\n📄 Aperçu des données nettoyées:")
print(wdi_wide.head())

# Sauvegarder temporairement
print("\n💾 Sauvegarde temporaire...")
wdi_final = wdi_wide.copy()
print("✅ wdi_final créé et prêt pour fusion !")
```

ANALYSE DE COMPLÉTUDE (2023)

Pays avec TOUTES les variables: 0

NaN par variable:

```
co2_per_capita      : 15 NaN ( 5.6%)
inflation           : 42 NaN (15.8%)
logistics_index     : 266 NaN (100.0%)
gdp_growth          : 15 NaN ( 5.6%)
gdp_per_capita      : 15 NaN ( 5.6%)
population          :  1 NaN ( 0.4%)
urban_pop_pct       :  1 NaN ( 0.4%)
tariff_rate         : 266 NaN (100.0%)
```

 Aperçu des données nettoyées:

Series Code	Country Name	Country Code	co2_per_capita \
0	Afghanistan	AFG	0.280995
1	Africa Eastern and Southern	AFE	0.838104
2	Africa Western and Central	AFW	0.494008
3	Albania	ALB	1.749144
4	Algeria	DZA	4.128275

Series Code	inflation	logistics_index	gdp_growth	gdp_per_capita \
0	-4.644709	NaN	2.266944	413.757895
1	7.399186	NaN	1.931160	1571.449189
2	5.221168	NaN	3.662428	1841.855064
3	4.758346	NaN	4.015417	9730.869219
4	9.322174	NaN	4.100000	5370.477235

Series Code	population	urban_pop_pct	tariff_rate
0	41454761.0	25.473053	NaN
1	750491370.0	37.772301	NaN
2	509398589.0	52.282908	NaN
3	2414095.0	58.210609	NaN
4	46164219.0	74.732808	NaN

 Sauvegarde temporaire...

 wdi_final créé et prêt pour fusion !

In [12]: `wdi_final.head(5)`

Out [12]:

Series Code	Country Name	Country Code	co2_per_capita	inflation	logistics_index	gdp_grow
0	Afghanistan	AFG	0.280995	-4.644709	NaN	2.2669
1	Africa Eastern and Southern	AFE	0.838104	7.399186	NaN	1.9311
2	Africa Western and Central	AFW	0.494008	5.221168	NaN	3.6624
3	Albania	ALB	1.749144	4.758346	NaN	4.0154
4	Algeria	DZA	4.128275	9.322174	NaN	4.1000

In [13]:

```

print("\n🇺🇦 ANALYSE DES NaN - WDI 2023")
print("="*80)

# Compter les NaN par colonne
print("\n❶ NaN par variable:")
for col in wdi_final.columns[2:]: # Skip Country Name et Code
    nan_count = wdi_final[col].isna().sum()
    total = len(wdi_final)
    nan_pct = (nan_count / total) * 100
    non_nan = total - nan_count
    print(f"{col:20s}: {non_nan:3d} valeurs / {total:3d} pays ({nan_pct:5.1f}%)")

# Pays avec TOUTES les 8 variables
print("\n❷ Pays avec données COMPLÈTES (0 NaN):")
complete = wdi_final.dropna()
print(f"Total: {len(complete)} pays")

# Pays avec au moins 6/8 variables
print("\n❸ Pays avec au moins 6/8 variables:")
max_nan_allowed = 2
pays_ok = wdi_final[wdi_final.isnull().sum(axis=1) <= max_nan_allowed]
print(f"Total: {len(pays_ok)} pays")

# Top 10 pays avec le plus de NaN
print("\n❹ Top 10 pays avec le PLUS de NaN:")
nan_counts = wdi_final.isnull().sum(axis=1).sort_values(ascending=False)
for country_idx, nan_count in nan_counts.items():
    country = wdi_final.loc[country_idx, 'Country Name']
    print(f" {country:40s}: {nan_count}/8 NaN")

print("\n" + "="*80)

```

ANALYSE DES NaN – WDI 2023

1 NaN par variable:

```
co2_per_capita      : 251 valeurs / 266 pays ( 5.6% NaN)
inflation           : 224 valeurs / 266 pays ( 15.8% NaN)
logistics_index     : 0 valeurs / 266 pays (100.0% NaN)
gdp_growth          : 251 valeurs / 266 pays ( 5.6% NaN)
gdp_per_capita      : 251 valeurs / 266 pays ( 5.6% NaN)
population          : 265 valeurs / 266 pays ( 0.4% NaN)
urban_pop_pct       : 265 valeurs / 266 pays ( 0.4% NaN)
tariff_rate         : 0 valeurs / 266 pays (100.0% NaN)
```

2 Pays avec données COMPLÈTES (0 NaN):

Total: 0 pays

3 Pays avec au moins 6/8 variables:

Total: 218 pays

4 Top 10 pays avec le PLUS de NaN:

```
Not classified      : 8/8 NaN
Isle of Man         : 6/8 NaN
St. Martin (French part) : 6/8 NaN
Guam                : 5/8 NaN
British Virgin Islands : 5/8 NaN
Korea, Dem. People's Rep. : 5/8 NaN
South Sudan         : 5/8 NaN
Eritrea             : 5/8 NaN
Northern Mariana Islands : 5/8 NaN
Syrian Arab Republic : 5/8 NaN
```

Nettoyage WGI

```
In [14]: wgi.head(20)
```

Out [14]:

	Country Name	Country Code	Series Name	Series Code	2018 [YR2018]	
0	Afghanistan	AFG	Control of Corruption: Estimate	CC.EST	-1.50287592411041	-
1	Afghanistan	AFG	Control of Corruption: Number of Sources	CC.NO.SRC	10	
2	Afghanistan	AFG	Control of Corruption: Percentile Rank	CC.PER.RNK	4.7619047164917	
3	Afghanistan	AFG	Control of Corruption: Percentile Rank, Lower ...	CC.PER.RNK.LOWER	0	
4	Afghanistan	AFG	Control of Corruption: Percentile Rank, Upper ...	CC.PER.RNK.UPPER	9.5238094329834	
5	Afghanistan	AFG	Control of Corruption: Standard Error	CC.STD.ERR	0.154826998710632	0
6	Afghanistan	AFG	Government Effectiveness: Estimate	GE.EST	-1.50175786018372	
7	Afghanistan	AFG	Government Effectiveness: Number of Sources	GE.NO.SRC	8	
8	Afghanistan	AFG	Government Effectiveness: Percentile Rank	GE.PER.RNK	7.14285707473755	1
9	Afghanistan	AFG	Government Effectiveness: Percentile Rank, Low...	GE.PER.RNK.LOWER	1.42857146263123	
10	Afghanistan	AFG	Government Effectiveness: Percentile Rank, Upp...	GE.PER.RNK.UPPER	11.9047622680664	
11	Afghanistan	AFG	Government Effectiveness: Standard Error	GE.STD.ERR	0.245759963989258	0.
12	Afghanistan	AFG	Political Stability and	PV.EST	-2.75344133377075	-

	Country Name	Country Code	Series Name	Series Code	2018 [YR2018]
			Absence of Violence/Te...		
13	Afghanistan	AFG	Political Stability and Absence of Violence/Te...	PV.NO.SRC	6
14	Afghanistan	AFG	Political Stability and Absence of Violence/Te...	PV.PER.RNK	0.471698105335236 C
15	Afghanistan	AFG	Political Stability and Absence of Violence/Te...	PV.PER.RNK.LOWER	0
16	Afghanistan	AFG	Political Stability and Absence of Violence/Te...	PV.PER.RNK.UPPER	2.35849046707153
17	Afghanistan	AFG	Political Stability and Absence of Violence/Te...	PV.STD.ERR	0.223692610859871
18	Afghanistan	AFG	Regulatory Quality: Estimate	RQ.EST	-1.13820362091064
19	Afghanistan	AFG	Regulatory Quality: Number of Sources	RQ.NO.SRC	9

```
In [15]: print("🧹 NETTOYAGE WGI")
print("="*80)

# ÉTAPE 1 : Voir tous les indicateurs disponibles
print("\n📊 Indicateurs WGI disponibles:")
print(wgi['Series Name'].unique())
print(f"\nTotal: {wgi['Series Name'].nunique()} indicateurs")

# ÉTAPE 2 : Filtrer uniquement les 2 indicateurs "Estimate" dont on a besoin
indicateurs_wgi = [
    'Political Stability and Absence of Violence/Terrorism: Estimate',
    'Control of Corruption: Estimate'
]

wgi_clean = wgi[wgi['Series Name'].isin(indicateurs_wgi)].copy()
print(f"\n✅ WGI filtré: {wgi_clean.shape}")
print(f"Indicateurs gardés: {wgi_clean['Series Name'].nunique()}/2")

# Vérification
print("\nIndicateurs gardés:")
print(wgi_clean['Series Name'].unique())

# ÉTAPE 3 : Convertir les valeurs en float (même problème que WDI)
```

```

year_cols = ['2018 [YR2018]', '2019 [YR2019]', '2020 [YR2020]',
             '2021 [YR2021]', '2022 [YR2022]', '2023 [YR2023]']

for col in year_cols:
    wgi_clean[col] = wgi_clean[col].replace('..', np.nan)
    wgi_clean[col] = pd.to_numeric(wgi_clean[col], errors='coerce')

print("\n✅ Conversion en numérique terminée")

# ÉTAPE 4 : Vérifier le taux de NaN par année
print("\n📊 Taux de NaN par année:")
for col in year_cols:
    nan_pct = (wgi_clean[col].isna().sum() / len(wgi_clean)) * 100
    valid = len(wgi_clean) - wgi_clean[col].isna().sum()
    print(f"{col}: {valid} valeurs, {nan_pct:.1f}% NaN")

# ÉTAPE 5 : Pivoter en format WIDE pour 2023
wgi_2023 = wgi_clean[['Country Name', 'Country Code', 'Series Code', '2023 [YR2023]']]

wgi_wide = wgi_2023.pivot(
    index=['Country Name', 'Country Code'],
    columns='Series Code',
    values='2023 [YR2023]'
).reset_index()

print(f"\n✅ Format WIDE créé: {wgi_wide.shape}")
print(f"Pays: {len(wgi_wide)}")
print(f"Variables: {len(wgi_wide.columns) - 2}")

# ÉTAPE 6 : Renommer avec noms courts
rename_wgi = {
    'PV.EST': 'political_stability',
    'CC.EST': 'control_corruption'
}

wgi_wide = wgi_wide.rename(columns=rename_wgi)

print("\n✅ Colonnes renommées")
print("Colonnes finales:")
print(wgi_wide.columns.tolist())

# ÉTAPE 7 : Analyse de complétude
print("\n" + "="*80)
print("📊 ANALYSE DE COMPLÉTUDE WGI (2023)")
print("="*80)

# NaN par variable
print("\nNaN par variable:")
for col in wgi_wide.columns[2:]:
    nan_count = wgi_wide[col].isna().sum()
    valid_count = len(wgi_wide) - nan_count
    nan_pct = (nan_count / len(wgi_wide)) * 100
    print(f"{col:25s}: {valid_count:3d} valeurs / {len(wgi_wide):3d} pays")

# Pays avec données complètes
complete = wgi_wide.dropna()
print(f"\nPays avec TOUTES les 2 variables WGI: {len(complete)}")

# ÉTAPE 8 : Aperçu
print("\n📄 Aperçu WGI nettoyé (5 premières lignes):")

```

```
print(wgi_wide.head())

# ÉTAPE 9 : Sauvegarder
wgi_final = wgi_wide.copy()
print("\n✅ wgi_final créé et prêt pour fusion !")

print("\n" + "="*80)
print("✅ WGI NETTOYAGE TERMINÉ")
print("="*80)
```

🧹 NETTOYAGE WGI

🇫🇷 Indicateurs WGI disponibles:

<StringArray>

```
[
'Control of Corruption: Estimate',
'Control of Corruption: Number of Sources',
'Control of Corruption: Percentile Rank',
'Control of Corruption: Percentile Rank,
Lower Bound of 90% Confidence Interval',
'Control of Corruption: Percentile Rank,
Upper Bound of 90% Confidence Interval',
'Control of Corruption: Standard Error',
'Government Effectiveness: Estimate',
'Government Effectiveness: Number of Sources',
'Government Effectiveness: Percentile Rank',
'Government Effectiveness: Percentile Rank,
Lower Bound of 90% Confidence Interval',
'Government Effectiveness: Percentile Rank,
Upper Bound of 90% Confidence Interval',
'Government Effectiveness: Standard Error',
'Political Stability and Absence of Violence/Terrorism: Estimate',
'Political Stability and Absence of Violence/Terrorism: Number of Sources',
'Political Stability and Absence of Violence/Terrorism: Percentile Rank',
'Political Stability and Absence of Violence/Terrorism: Percentile Rank,
Lower Bound of 90% Confidence Interval',
'Political Stability and Absence of Violence/Terrorism: Percentile Rank,
Upper Bound of 90% Confidence Interval',
'Political Stability and Absence of Violence/Terrorism: Standard Error',
'Regulatory Quality: Estimate',
'Regulatory Quality: Number of Sources',
'Regulatory Quality: Percentile Rank',
'Regulatory Quality: Percentile Rank,
Lower Bound of 90% Confidence Interval',
'Regulatory Quality: Percentile Rank,
Upper Bound of 90% Confidence Interval',
'Regulatory Quality: Standard Error',
'Rule of Law: Estimate',
'Rule of Law: Number of Sources',
'Rule of Law: Percentile Rank',
```

```

Lower Bound of 90% Confidence Interval',
Upper Bound of 90% Confidence Interval',
'Rule of Law: Standard Error',
'Voice and Accountability: Estimate',
and Accountability: Number of Sources',
ce and Accountability: Percentile Rank',
Lower Bound of 90% Confidence Interval',
Upper Bound of 90% Confidence Interval',
ice and Accountability: Standard Error',
nan]
Length: 37, dtype: str

```

Total: 36 indicateurs

✓ WGI filtré: (428, 10)
Indicateurs gardés: 2/2

Indicateurs gardés:
<StringArray>
['Control of Corruption: Estimate', 'Political Stability and Absence of Violence/Terrorism: Estimate']
Length: 2, dtype: str

✓ Conversion en numérique terminée

🇫🇷 Taux de NaN par année:
2018 [YR2018]: 424 valeurs, 0.9% NaN
2019 [YR2019]: 424 valeurs, 0.9% NaN
2020 [YR2020]: 424 valeurs, 0.9% NaN
2021 [YR2021]: 424 valeurs, 0.9% NaN
2022 [YR2022]: 426 valeurs, 0.5% NaN
2023 [YR2023]: 425 valeurs, 0.7% NaN

✓ Format WIDE créé: (214, 4)
Pays: 214
Variables: 2

✓ Colonnes renommées
Colonnes finales:
['Country Name', 'Country Code', 'control_corruption', 'political_stability']

```

=====
=====
🇫🇷 ANALYSE DE COMPLÉTUDE WGI (2023)
=====
=====

```

NaN par variable:
control_corruption : 213 valeurs / 214 pays (0.5% NaN)

political_stability : 212 valeurs / 214 pays (0.9% NaN)

Pays avec TOUTES les 2 variables WGI: 212

 Aperçu WGI nettoyé (5 premières lignes):

Series Code	Country Name	Country Code	control_corruption \
0	Afghanistan	AFG	-1.154932
1	Albania	ALB	-0.332219
2	Algeria	DZA	-0.589308
3	American Samoa	ASM	1.251356
4	Andorra	AND	1.251356

Series Code	political_stability
0	-2.484081
1	0.183349
2	-0.577885
3	1.114221
4	1.583466

✅ wgi_final créé et prêt pour fusion !

=====

=====

✅ WGI NETTOYAGE TERMINÉ

=====

=====

```
In [16]: print("\n NETTOYAGE CEPII - DISTANCES")
print("\n*80)

# Structure
print(f"Shape: {dist_cepii.shape}")
print(f"\nColonnes:")
print(dist_cepii.columns.tolist())

# Filtrer uniquement distances DEPUIS la France
france_dist = dist_cepii[dist_cepii['iso_o'] == 'FRA'].copy()
print(f"\n✅ Distances depuis France: {len(france_dist)} pays")

# Garder uniquement les colonnes utiles
cepii_final = france_dist[['iso_d', 'distw']].copy()

# CORRECTION : Convertir distw en float
cepii_final['distw'] = pd.to_numeric(cepii_final['distw'], errors='coerce')

# Renommer
cepii_final.columns = ['Country Code', 'distance_from_france']

print(f"\n✅ CEPII nettoyé: {cepii_final.shape}")
print("\nAperçu (10 premiers pays):")
print(cepii_final.head(10))

# Vérifier le type
print(f"\nType de distance_from_france: {cepii_final['distance_from_france'].dtype}")

# Stats (maintenant ça devrait marcher)
print(f"\n📊 Statistiques:")
print(f"Distance min: {cepii_final['distance_from_france'].min():.0f} km")
print(f"Distance max: {cepii_final['distance_from_france'].max():.0f} km")
print(f"Distance moyenne: {cepii_final['distance_from_france'].mean():.0f} km")
```

```
print(f"NaN: {cepii_final['distance_from_france'].isna().sum()}")

# Quelques exemples de pays
print("\n🌐 Exemples de distances:")
exemples = cepii_final.head(20)
for idx, row in exemples.iterrows():
    print(f"{row['Country Code']:3s}: {row['distance_from_france']:>8.0f}")

print("\n✅ cepii_final créé !")
```

NETTOYAGE CEPII – DISTANCES

=====

=====

Shape: (50176, 14)

Colonnes:

['iso_o', 'iso_d', 'contig', 'comlang_off', 'comlang_ethno', 'colony', 'comcol', 'curcol', 'col45', 'smctry', 'dist', 'distcap', 'distw', 'distwce', 's']

✅ Distances depuis France: 224 pays

✅ CEPII nettoyé: (224, 2)

Aperçu (10 premiers pays):

	Country Code	distance_from_france
15008	ABW	7716.7520
15009	AFG	5500.6650
15010	AGO	6384.8050
15011	AIA	6743.4270
15012	ALB	1499.4510
15013	AND	546.5107
15014	ANT	7541.4990
15015	ARE	5180.4300
15016	ARG	10932.3400
15017	ARM	3402.5700

Type de distance_from_france: float64

 Statistiques:

Distance min: 409 km

Distance max: 18894 km

Distance moyenne: 6553 km

NaN: 5

 Exemples de distances:

ABW:	7717 km
AFG:	5501 km
AGO:	6385 km
AIA:	6743 km
ALB:	1499 km
AND:	547 km
ANT:	7541 km
ARE:	5180 km
ARG:	10932 km
ARM:	3403 km
ATG:	6739 km
AUS:	16513 km
AUT:	976 km
AZE:	3720 km
BDI:	6152 km
BEL:	526 km
BEN:	4343 km
BFA:	3882 km
BGD:	7944 km
BGR:	1802 km

✅ cepii_final créé !

```
In [17]: print("🌿 NETTOYAGE FAO – FOOD BALANCE")
print("="*80)

# Structure
print(f"Shape: {food_balance.shape}")
print(f"Colonnes: {food_balance.columns.tolist()}")
print(f"\nPremières lignes:")
print(food_balance.head())

# Items disponibles avec "meat"
print("\n🔍 Items avec 'MEAT':")
meat_items = food_balance['Item'].unique()
meat_items_filtered = [item for item in meat_items if 'meat' in str(item)]
for idx, item in enumerate(meat_items_filtered[:10], 1):
    print(f" {idx}. {item}")

# Elements disponibles
print("\n🔍 Elements disponibles:")
elements = food_balance['Element'].unique()
for idx, element in enumerate(elements[:15], 1):
    print(f" {idx}. {element}")

# Années disponibles
print(f"\n📅 Années: {food_balance['Year'].min()} à {food_balance['Year'].max()}")

print("\n" + "="*80)
print("Quelle combinaison Item + Element veux-tu ?")
print("Suggestion : Item='Meat' + Element='Food supply quantity (kg/capit
```

NETTOYAGE FAO – FOOD BALANCE

=====

Shape: (4820497, 14)

Colonnes: ['Area Code', 'Area Code (M49)', 'Area', 'Item Code', 'Item Code (FBS)', 'Item', 'Element Code', 'Element', 'Year Code', 'Year', 'Unit', 'Value', 'Flag', 'Note']

Premières lignes:

	Area Code	Area Code (M49)	Area	Item Code	Item Code (FBS)	\
0	2	'004	Afghanistan	2501	'S2501	
1	3	'008	Albania	2501	'S2501	
2	2	'004	Afghanistan	2501	'S2501	
3	3	'008	Albania	2501	'S2501	
4	2	'004	Afghanistan	2501	'S2501	

r \	Item	Element Code	Element	Year Code	Year
0	Population	511	Total Population – Both sexes	2010	201
0					
1	Population	511	Total Population – Both sexes	2010	201
0					
2	Population	511	Total Population – Both sexes	2011	201
1					
3	Population	511	Total Population – Both sexes	2011	201
1					
4	Population	511	Total Population – Both sexes	2012	201
2					

	Unit	Value	Flag	Note
0	1000 No	28284.09	X	NaN
1	1000 No	2928.72	X	NaN
2	1000 No	29347.71	X	NaN
3	1000 No	2911.50	X	NaN
4	1000 No	30560.03	X	NaN

 Items avec 'MEAT':

1. Meat
2. Bovine Meat
3. Mutton & Goat Meat
4. Pigmeat
5. Poultry Meat
6. Meat, Other
7. Meat, Aquatic Mammals

 Elements disponibles:

1. Total Population – Both sexes
2. Food supply (kcal/capita/day)
3. Food supply (kcal)
4. Protein supply quantity (g/capita/day)
5. Protein supply quantity (t)
6. Fat supply quantity (g/capita/day)
7. Fat supply quantity (t)
8. Production
9. Import quantity
10. Stock Variation
11. Export quantity
12. Domestic supply quantity
13. Feed
14. Losses

15. Other uses (non-food)

 Années: 2010 à 2023

=====

=====

Quelle combinaison Item + Element veux-tu ?

Suggestion : Item='Meat' + Element='Food supply quantity (kg/capita/yr)'

```
In [18]: print("🔪 FILTRAGE FOOD BALANCE – Consommation de viande")
print("="*80)

# Filtrer : Item='Meat' + Element='Food supply (kcal/capita/day)'
fao_food_clean = food_balance[
    (food_balance['Item'] == 'Meat') &
    (food_balance['Element'] == 'Food supply (kcal/capita/day)')
].copy()

print(f"✅ Après filtrage: {fao_food_clean.shape}")
print(f"Pays uniques: {fao_food_clean['Area'].nunique()}")
print(f"Années: {fao_food_clean['Year'].min()} à {fao_food_clean['Year'].max()}")

# Vérifier les données pour 2023
fao_2023 = fao_food_clean[fao_food_clean['Year'] == 2023].copy()
print(f"\n📊 Données 2023: {len(fao_2023)} pays")

# Convertir Value en numérique
fao_2023['Value'] = pd.to_numeric(fao_2023['Value'], errors='coerce')

# Créer le dataset final
fao_food_final = fao_2023[['Area', 'Value']].copy()
fao_food_final.columns = ['Area', 'meat_consumption_kcal_day']

# Statistiques
print(f"\n📊 Statistiques (2023):")
print(f"NaN: {fao_food_final['meat_consumption_kcal_day'].isna().sum()}")
print(f"Min: {fao_food_final['meat_consumption_kcal_day'].min():.1f} kcal")
print(f"Max: {fao_food_final['meat_consumption_kcal_day'].max():.1f} kcal")
print(f"Moyenne: {fao_food_final['meat_consumption_kcal_day'].mean():.1f} kcal")

print("\n📄 Aperçu (10 premiers pays):")
print(fao_food_final.head(10))

# ⚠️ PROBLÈME : Les noms de pays FAO != codes ISO3 de WDI
print("\n⚠️ ATTENTION: Les noms FAO ne correspondent pas aux codes ISO3")
print("Exemples de noms FAO:")
print(fao_food_final['Area'].head(20).tolist())

print("\n✅ fao_food_final créé (mais il faudra harmoniser les codes pays")
```

FILTRAGE FOOD BALANCE – Consommation de viande

=====

✓ Après filtrage: (2901, 14)

Pays uniques: 213

Années: 2010 à 2023

 Données 2023: 211 pays

 Statistiques (2023):

NaN: 0

Min: 15.1 kcal/day

Max: 755.0 kcal/day

Moyenne: 251.1 kcal/day

 Aperçu (10 premiers pays):

	Area	meat_consumption_kcal_day
874004	Micronesia (Federated States of)	288.78
884828	Tuvalu	391.27
947623	Nauru	509.60
1103448	Tonga	755.00
1112983	Marshall Islands	539.36
1219710	Bhutan	66.47
1230566	Qatar	296.37
1302554	Bahrain	349.96
2664812	Turkmenistan	271.83
2741352	Timor-Leste	157.57

⚠ ATTENTION: Les noms FAO ne correspondent pas aux codes IS03

Exemples de noms FAO:

['Micronesia (Federated States of)', 'Tuvalu', 'Nauru', 'Tonga', 'Marshall Islands', 'Bhutan', 'Qatar', 'Bahrain', 'Turkmenistan', 'Timor-Leste', 'Lesotho', 'Kiribati', 'Guinea-Bissau', 'Comoros', 'Djibouti', 'Sao Tome and Principe', 'Liberia', 'Mauritania', 'Micronesia', 'Tajikistan']

✓ fao_food_final créé (mais il faudra harmoniser les codes pays !)

```
In [48]: print( JUSTIFICATION UNITÉ – meat_consumption_kcal_day")
print("=*80)
print("""
Unité retenue : kcal/capita/day (énergie calorique journalière issue de l

Justification :
- Disponible directement dans FAO Food Balance Sheet (Element = 'Food sup
- Capture la réalité de la consommation effective (≠ production ou dispon
- Permet des comparaisons inter-pays indépendamment des différences de co
des régimes alimentaires (viande dense vs viande maigre)
- Corrélée avec kg/capita/an mais plus robuste aux variations de type de

Note : 1 kg viande ≈ 1500–3000 kcal selon l'espèce → l'ordre de grandeur
est cohérent (moyenne mondiale ~251 kcal/j ≈ 80–170 kg/an selon espèce)
""")
```

🇫🇷 JUSTIFICATION UNITÉ – meat_consumption_kcal_day

Unité retenue : kcal/capita/day (énergie calorique journalière issue de la viande)

Justification :

- Disponible directement dans FAO Food Balance Sheet (Element = 'Food supply (kcal/capita/day)')
- Capture la réalité de la consommation effective (≠ production ou disponibilité)
- Permet des comparaisons inter-pays indépendamment des différences de composition des régimes alimentaires (viande dense vs viande maigre)
- Corrélée avec kg/capita/an mais plus robuste aux variations de type de viande

Note : 1 kg viande ≈ 1500–3000 kcal selon l'espèce → l'ordre de grandeur est cohérent (moyenne mondiale ~251 kcal/j ≈ 80–170 kg/an selon espèce)

```
In [19]: print("🔍 VÉRIFICATION DES CODES PAYS FAO")
print("="*80)

# Vérifier les colonnes de codes
print("Colonnes disponibles dans food_balance:")
print(food_balance.columns.tolist())

# Regarder un échantillon avec les codes
sample = food_balance[
    (food_balance['Item'] == 'Meat') &
    (food_balance['Element'] == 'Food supply (kcal/capita/day)') &
    (food_balance['Year'] == 2023)
][['Area Code', 'Area Code (M49)', 'Area']].head(20)

print("\n📄 Échantillon codes FAO:")
print(sample)

# Vérifier s'il existe une correspondance ISO3
# Normalement FAO a un code "M49" (code numérique UN)
# qu'on peut convertir en ISO3
```


🔍 VÉRIFICATION DES CODES PAYS FAO

=====

=====

Colonnes disponibles dans food_balance:

['Area Code', 'Area Code (M49)', 'Area', 'Item Code', 'Item Code (FBS)', 'Item', 'Element Code', 'Element', 'Year Code', 'Year', 'Unit', 'Value', 'Flag', 'Note']

📄 Échantillon codes FAO:

	Area Code	Area Code (M49)	Area
874004	145	'583	Micronesia (Federated States of)
884828	227	'798	Tuvalu
947623	148	'520	Nauru
1103448	219	'776	Tonga
1112983	127	'584	Marshall Islands
1219710	18	'064	Bhutan
1230566	179	'634	Qatar
1302554	13	'048	Bahrain
2664812	213	'795	Turkmenistan
2741352	176	'626	Timor-Leste
2780549	122	'426	Lesotho
2781004	83	'296	Kiribati
2903892	175	'624	Guinea-Bissau
2914073	45	'174	Comoros
2932135	72	'262	Djibouti
2972645	193	'678	Sao Tome and Principe
3008198	123	'430	Liberia
3011226	136	'478	Mauritania
3014646	5503	'057	Micronesia
3021847	208	'762	Tajikistan

```
In [20]: print("🚀 CRÉATION TABLE DE CORRESPONDANCE FAO → ISO3")
print("="*80)

# Option 1 : Utiliser pycountry (package Python)
try:
    import pycountry
    print("✅ pycountry installé")
except:
    print("⚠️ pycountry non installé – installation nécessaire")
    print("Lance : pip install pycountry")

# Fonction de mapping manuel pour les cas spéciaux FAO
def fao_to_iso3(fao_name):
    """Convertir nom FAO en code ISO3"""

    # Mapping manuel pour cas spéciaux FAO
    special_cases = {
        'United States of America': 'USA',
        'United Kingdom of Great Britain and Northern Ireland': 'GBR',
        'China, mainland': 'CHN',
        'China, Hong Kong SAR': 'HKG',
        'China, Macao SAR': 'MAC',
        'China, Taiwan Province of': 'TWN',
        'Republic of Korea': 'KOR',
        'Democratic People's Republic of Korea': 'PRK',
        'Republic of Moldova': 'MDA',
        'Russian Federation': 'RUS',
        'Bolivia (Plurinational State of)': 'BOL',
        'Venezuela (Bolivarian Republic of)': 'VEN',
    }
```

```

        'Iran (Islamic Republic of)': 'IRN',
        'Syrian Arab Republic': 'SYR',
        'Viet Nam': 'VNM',
        'Côte d'Ivoire': 'CIV',
        'Czechia': 'CZE',
        'Türkiye': 'TUR',
        'United Republic of Tanzania': 'TZA',
        'Micronesia (Federated States of)': 'FSM',
        'Congo': 'COG',
        'Democratic Republic of the Congo': 'COD',
    }

    if fao_name in special_cases:
        return special_cases[fao_name]

    # Essayer pycountry pour le reste
    try:
        country = pycountry.countries.search_fuzzy(fao_name)[0]
        return country.alpha_3
    except:
        return None

# Tester sur quelques pays
test_countries = fao_food_final['Area'].head(20).unique()
print("\n🔪 Test de conversion:")
for country in test_countries:
    iso3 = fao_to_iso3(country)
    country_str = str(country)[:45]
    print(f"{country_str:45s} → {iso3}")

# Appliquer à tout le dataset
print("\n🔄 Application à tout le dataset...")
fao_food_final['Country Code'] = fao_food_final['Area'].apply(fao_to_iso3)

# Vérifier combien ont été convertis
converted = fao_food_final['Country Code'].notna().sum()
total = len(fao_food_final)
pct = (converted/total)*100
print(f"\n✅ Conversion réussie: {converted}/{total} pays ({pct:.1f}%)")

# Voir les pays non convertis
non_converted = fao_food_final[fao_food_final['Country Code'].isna()]['Ar
if len(non_converted) > 0:
    print(f"\n⚠️ Pays non convertis ({len(non_converted)}):")
    for country in non_converted:
        print(f" - {country}")

print("\n📄 Aperçu final:")
print(fao_food_final[['Country Code', 'Area', 'meat_consumption_kcal_day'

print("\n✅ fao_food_final prêt avec codes ISO3 !")

```

CRÉATION TABLE DE CORRESPONDANCE FAO → IS03

=====

=====

✅ pycountry installé

 Test de conversion:

Micronesia (Federated States of)	→ FSM
Tuvalu	→ TUV
Nauru	→ NRU
Tonga	→ TON
Marshall Islands	→ MHL
Bhutan	→ BTN
Qatar	→ QAT
Bahrain	→ BHR
Turkmenistan	→ TKM
Timor-Leste	→ TLS
Lesotho	→ LSO
Kiribati	→ KIR
Guinea-Bissau	→ GNB
Comoros	→ COM
Djibouti	→ DJI
Sao Tome and Principe	→ STP
Liberia	→ LBR
Mauritania	→ MRT
Micronesia	→ FSM
Tajikistan	→ TJK

 Application à tout le dataset...

✅ Conversion réussie: 179/211 pays (84.8%)

⚠ Pays non convertis (32):

- Netherlands (Kingdom of the)
- Central Asia
- Northern America
- Australia and New Zealand
- Melanesia
- Western Europe
- Northern Europe
- Eastern Europe
- Southern Africa
- Middle Africa
- South America
- Northern Africa
- Western Africa
- Caribbean
- Oceania
- Southern Europe
- South-eastern Asia
- Central America
- Eastern Asia
- Eastern Africa
- Southern Asia
- Small Island Developing States (SIDS)
- Western Asia
- European Union (27)
- Europe
- Least Developed Countries (LDCs)
- Land Locked Developing Countries (LLDCs)
- Low Income Food Deficit Countries (LIFDCs)

- Americas
- Net Food Importing Developing Countries (NFIDCs)
- Asia
- World

📄 Aperçu final:

	Country Code	Area \
874004	FSM	Micronesia (Federated States of)
884828	TUV	Tuvalu
947623	NRU	Nauru
1103448	TON	Tonga
1112983	MHL	Marshall Islands
1219710	BTN	Bhutan
1230566	QAT	Qatar
1302554	BHR	Bahrain
2664812	TKM	Turkmenistan
2741352	TLS	Timor-Leste

	meat_consumption_kcal_day
874004	288.78
884828	391.27
947623	509.60
1103448	755.00
1112983	539.36
1219710	66.47
1230566	296.37
1302554	349.96
2664812	271.83
2741352	157.57

✅ fao_food_final prêt avec codes IS03 !

```
In [21]: # Supprimer les agrégats régionaux (garder uniquement les vrais pays)
fao_food_final = fao_food_final[fao_food_final['Country Code'].notna()]

print(f"✅ FAO Food Balance finalisé: {len(fao_food_final)} pays avec codes IS03")
print(fao_food_final[['Country Code', 'Area', 'meat_consumption_kcal_day']])
```

✅ FAO Food Balance finalisé: 179 pays avec codes IS03

	Country Code	Area \
874004	FSM	Micronesia (Federated States of)
884828	TUV	Tuvalu
947623	NRU	Nauru
1103448	TON	Tonga
1112983	MHL	Marshall Islands

	meat_consumption_kcal_day
874004	288.78
884828	391.27
947623	509.60
1103448	755.00
1112983	539.36

```
In [22]: print("\n🌐 NETTOYAGE FAO – TRADE DETAILED (Importations volaille)")
print("="*80)

# Structure
print(f"Shape: {trade_detailed.shape}")
print(f"Colonnes: {trade_detailed.columns.tolist()}")
```

```
# Items avec "chicken" ou "poultry"
print("\n🔍 Items avec 'CHICKEN' ou 'POULTRY':")
items = trade_detailed['Item'].unique()
poultry_items = [item for item in items if 'chicken' in str(item).lower()]
for idx, item in enumerate(poultry_items, 1):
    print(f" {idx}. {item}")

# Elements disponibles
print("\n🔍 Elements disponibles:")
elements = trade_detailed['Element'].unique()
for idx, element in enumerate(elements[:15], 1):
    print(f" {idx}. {element}")

# Années disponibles
print(f"\n📅 Années: {trade_detailed['Year'].min()} à {trade_detailed['Year'].max()}")

print("\n" + "="*80)
print("Quelle combinaison ?")
print("Suggestion : Item='Meat of chickens, fresh or chilled' + Element='Import quantity'")
print("Ou : Item='Meat, chicken' si ça existe")
```

🌐 NETTOYAGE FAO – TRADE DETAILED (Importations volaille)

```
=====
=====
Shape: (52410630, 16)
Colonnes: ['Reporter Country Code', 'Reporter Country Code (M49)', 'Reporter Countries', 'Partner Country Code', 'Partner Country Code (M49)', 'Partner Countries', 'Item Code', 'Item Code (CPC)', 'Item', 'Element Code', 'Element', 'Year Code', 'Year', 'Unit', 'Value', 'Flag']
```

🔍 Items avec 'CHICKEN' ou 'POULTRY':

1. Poultry meat preparations
2. Meat of chickens, fresh or chilled
3. Chickens
4. Edible offals and liver of chickens and guinea fowl, fresh, chilled or frozen
5. Feed, compound, poultry
6. Poultry fat, rendered
7. Fat of poultry

🔍 Elements disponibles:

1. Import quantity
2. Import value
3. Export quantity
4. Export value

📅 Années: 1986 à 2024

```
=====
=====
Quelle combinaison ?
Suggestion : Item='Meat of chickens, fresh or chilled' + Element='Import quantity'
Ou : Item='Meat, chicken' si ça existe
```

```
In [23]: print("🔪 FILTRAGE TRADE DETAILED – Importations de poulet (2 items)")
print("="*80)

# Filtrer les 2 items + Import quantity
fao_trade_clean = trade_detailed[
```

```

    (trade_detailed['Item'].isin(['Meat of chickens, fresh or chilled', '
    (trade_detailed['Element'] == 'Import quantity')
].copy()

print(f"✅ Après filtrage: {fao_trade_clean.shape}")
print(f"Pays rapporteurs: {fao_trade_clean['Reporter Countries'].nunique()")
print(f"Items: {fao_trade_clean['Item'].nunique()}")
print(f"Années: {fao_trade_clean['Year'].min()} à {fao_trade_clean['Year']

# Vérifier la structure
print("\n🇺🇦 Structure (5 premières lignes):")
print(fao_trade_clean[['Reporter Countries', 'Partner Countries', 'Item',

# Vérifier si "World" existe comme partenaire
partners = fao_trade_clean['Partner Countries'].unique()
print(f"\nNombre de partenaires uniques: {len(partners)}")
print("\nPremiers 20 partenaires:")
print(partners[:20])

# Vérifier "World"
world_exists = 'World' in partners
print(f"\n{'✅' if world_exists else '❌'} 'World' existe comme partenair

if world_exists:
    print("\n→ On filtrera Partner='World' pour avoir les imports totaux

    # Filtrer World + année 2023
    fao_trade_world = fao_trade_clean[
        (fao_trade_clean['Partner Countries'] == 'World') &
        (fao_trade_clean['Year'] == 2023)
    ].copy()

    print(f"\n🇺🇦 Données 2023 (Partner=World): {len(fao_trade_world)} lig
    print(f"Pays: {fao_trade_world['Reporter Countries'].nunique()}")
    print(f"Items: {fao_trade_world['Item'].nunique()}")

```

🔧 FILTRAGE TRADE DETAILED – Importations de poulet (2 items)

=====

✅ Après filtrage: (79041, 16)

Pays rapporteurs: 195

Items: 2

Années: 1986 à 2024

🇺🇦 Structure (5 premières lignes):

	Reporter Countries	Partner Countries	\
27	Afghanistan	Argentina	
875	Bhutan	Australia	
923	Bhutan	Australia	
1298	Democratic Republic of the Congo	Afghanistan	
1321	Congo	Albania	

	Item	Year	Value	Unit
27	Meat of chickens, fresh or chilled	2011	198.00	t
875	Chickens	2010	2.00	1000 An
923	Chickens	2012	3.00	1000 An
1298	Meat of chickens, fresh or chilled	2021	13.81	t
1321	Meat of chickens, fresh or chilled	2021	23.73	t

Nombre de partenaires uniques: 206

Premiers 20 partenaires:

<StringArray>

```
[
    'Argentina',
    'Albania',
    'Austria',
    'Bangladesh',
    'Bulgaria',
    'Angola',
    'Bahamas',
    'Australia',
    'Antigua and Barbuda',
    'Belarus',
    'Belgium-Luxembourg',
    'Canada',
    'Armenia',
    'Cambodia',
    'Afghanistan',
    'Algeria',
    'Belgium',
    'Brazil',
    'China, mainland',
    'Barbados',
]
```

Length: 20, dtype: str

❌ 'World' existe comme partenaire

```
In [24]: print("🔄 AGRÉGATION DES IMPORTS – Somme par pays rapporteur")
print("="*80)

# Filtrer année 2023
fao_trade_2023 = fao_trade_clean[fao_trade_clean['Year'] == 2023].copy()

print(f"Données 2023: {len(fao_trade_2023)} lignes")

# Convertir Value en numérique
fao_trade_2023['Value'] = pd.to_numeric(fao_trade_2023['Value'], errors='coerce')

# Vérifier les unités
print("\nUnités utilisées:")
print(fao_trade_2023[['Item', 'Unit']].drop_duplicates())

# PROBLÈME : Les 2 items ont des unités différentes !
# - "Meat of chickens" → tonnes (t)
# - "Chickens" → 1000 animaux (1000 An)
# On ne peut pas les additionner directement !

print("\n⚠️ ATTENTION : Unités différentes")
print("→ 'Meat of chickens' = tonnes")
```

```

print("→ 'Chickens' = 1000 animaux")
print("\nSOLUTIONS:")
print("1. Garder uniquement 'Meat of chickens' (viande)")
print("2. Créer 2 variables séparées")
print("3. Convertir poulets vivants en équivalent tonnes (approximatif)")

# OPTION 1 : Garder uniquement la VIANDE (RECOMMANDÉ)
print("\n" + "="*80)
print("OPTION 1 : Garder uniquement 'Meat of chickens' (RECOMMANDÉ)")
print("="*80)

fao_trade_meat = fao_trade_2023[
    fao_trade_2023['Item'] == 'Meat of chickens, fresh or chilled'
].copy()

# Agréger par pays rapporteur (somme des imports de tous les partenaires)
fao_trade_agg = fao_trade_meat.groupby('Reporter Countries')['Value'].sum
fao_trade_agg.columns = ['Area', 'chicken_imports_tonnes']

print(f"\n✅ Agrégation terminée: {len(fao_trade_agg)} pays")
print(f"NaN: {fao_trade_agg['chicken_imports_tonnes'].isna().sum()}")

# Stats
print("\n📊 Statistiques:")
print(f"Min: {fao_trade_agg['chicken_imports_tonnes'].min():.1f} tonnes")
print(f"Max: {fao_trade_agg['chicken_imports_tonnes'].max():.1f} tonnes")
print(f"Moyenne: {fao_trade_agg['chicken_imports_tonnes'].mean():.1f} ton")

print("\n📦 Top 10 importateurs:")
top10 = fao_trade_agg.nlargest(10, 'chicken_imports_tonnes')
print(top10)

# Appliquer la conversion FAO → ISO3
print("\n🌐 Conversion FAO → ISO3...")
fao_trade_agg['Country Code'] = fao_trade_agg['Area'].apply(fao_to_iso3)

converted = fao_trade_agg['Country Code'].notna().sum()
total = len(fao_trade_agg)
print(f"✅ Conversion: {converted}/{total} pays ({(converted/total)*100:.1f}%)")

# Supprimer les agrégats
fao_trade_final = fao_trade_agg[fao_trade_agg['Country Code'].notna()].copy()

print(f"\n✅ fao_trade_final créé: {len(fao_trade_final)} pays")
print("\n📋 Aperçu:")
print(fao_trade_final[['Country Code', 'Area', 'chicken_imports_tonnes']])

```


🔄 AGRÉGATION DES IMPORTS – Somme par pays rapporteur

=====

=====

Données 2023: 3472 lignes

Unités utilisées:

	Item	Unit
1429	Meat of chickens, fresh or chilled	t
50185	Chickens	1000 An
50577	Chickens	t

⚠ ATTENTION : Unités différentes

→ 'Meat of chickens' = tonnes

→ 'Chickens' = 1000 animaux

SOLUTIONS:

1. Garder uniquement 'Meat of chickens' (viande)
2. Créer 2 variables séparées
3. Convertir poulets vivants en équivalent tonnes (approximatif)

=====

=====

OPTION 1 : Garder uniquement 'Meat of chickens' (RECOMMANDÉ)

=====

=====

✅ Agrégation terminée: 147 pays

NaN: 0

📊 Statistiques:

Min: 0.0 tonnes

Max: 1285570.4 tonnes

Moyenne: 83946.6 tonnes

📋 Top 10 importateurs:

	Area	chicken_imports_to
nnes		
30	China, mainland	128557
0.36		
87	Mexico	96711
2.39		
93	Netherlands (Kingdom of the)	65262
0.63		
69	Japan	58485
2.38		
50	France	49525
3.05		
137	United Arab Emirates	49489
3.10		
54	Germany	49294
1.64		
138	United Kingdom of Great Britain and Northern I...	45169
7.55		
114	Saudi Arabia	39804
8.44		
121	South Africa	31278
9.34		

🌐 Conversion FAO → IS03...

✅ Conversion: 146/147 pays (99.3%)

✅ fao_trade_final créé: 146 pays

📄 Aperçu:

	Country Code	Area	chicken_imports_tonnes
0	ALB	Albania	30096.41
1	AGO	Angola	236746.25
2	ATG	Antigua and Barbuda	6223.53
3	ARG	Argentina	1715.68
4	ARM	Armenia	38210.90
5	AUS	Australia	69.79
6	AUT	Austria	56823.69
7	AZE	Azerbaijan	25822.72
8	BHS	Bahamas	17811.13
9	BHR	Bahrain	48725.19

```
In [25]: print("🇫🇷 RÉCAPITULATIF COMPLET DE TES INDICATEURS")
print("="*80)

# 1. WDI
print("\n📄 WDI - WORLD DEVELOPMENT INDICATORS (8 variables)")
print("-"*80)
wdi_vars = [col for col in wdi_final.columns if col not in ['Country Name']]
for idx, var in enumerate(wdi_vars, 1):
    print(f"    {idx}. {var}")

# 2. WGI
print("\n📄 WGI - WORLDWIDE GOVERNANCE INDICATORS (2 variables)")
print("-"*80)
wgi_vars = [col for col in wgi_final.columns if col not in ['Country Name']]
for idx, var in enumerate(wgi_vars, 1):
    print(f"    {idx}. {var}")

# 3. CEPII
print("\n📄 CEPII - DISTANCES GÉOGRAPHIQUES (1 variable)")
print("-"*80)
cepii_vars = [col for col in cepii_final.columns if col != 'Country Code']
for idx, var in enumerate(cepii_vars, 1):
    print(f"    {idx}. {var}")

# 4. FAO Food Balance
print("\n📄 FAO - FOOD BALANCE (1 variable)")
print("-"*80)
fao_food_vars = [col for col in fao_food_final.columns if col not in ['Co']]
for idx, var in enumerate(fao_food_vars, 1):
    print(f"    {idx}. {var}")

# 5. FAO Trade
print("\n📄 FAO - TRADE DETAILED (1 variable)")
print("-"*80)
fao_trade_vars = [col for col in fao_trade_final.columns if col not in ['']]
for idx, var in enumerate(fao_trade_vars, 1):
    print(f"    {idx}. {var}")

# TOTAL
print("\n" + "="*80)
print("✅ TOTAL DES INDICATEURS")
print("="*80)
total = len(wdi_vars) + len(wgi_vars) + len(cepii_vars) + len(fao_food_va)
print(f"TOTAL: {total} variables")
```

```

# Répartition PESTEL
print("\n🌀 RÉPARTITION PESTEL")
print("="*80)
print("P - Politique           : 2 variables (political_stability, contro
print("E - Économique         : 4 variables (gdp_per_capita, gdp_growth,
print("S - Social              : 3 variables (population, urban_pop_pct,
print("T - Technologique       : 2 variables (logistics_index, distance_f
print("E - Environnemental    : 1 variable (co2_per_capita)")
print("L - Légal              : 1 variable (tariff_rate)")

# Liste complète alphabétique
print("\n📁 LISTE COMPLÈTE ALPHABÉTIQUE")
print("="*80)
all_vars = wdi_vars + wgi_vars + cepii_vars + fao_food_vars + fao_trade_v
all_vars_sorted = sorted(all_vars)
for idx, var in enumerate(all_vars_sorted, 1):
    print(f"{idx:2d}. {var}")

```

RÉCAPITULATIF COMPLET DE TES INDICATEURS

=====

=====

WDI – WORLD DEVELOPMENT INDICATORS (8 variables)

1. co2_per_capita
2. inflation
3. logistics_index
4. gdp_growth
5. gdp_per_capita
6. population
7. urban_pop_pct
8. tariff_rate

WGI – WORLDWIDE GOVERNANCE INDICATORS (2 variables)

1. control_corruption
2. political_stability

CEPII – DISTANCES GÉOGRAPHIQUES (1 variable)

1. distance_from_france

FAO – FOOD BALANCE (1 variable)

1. meat_consumption_kcal_day

FAO – TRADE DETAILED (1 variable)

1. chicken_imports_tonnes

=====

=====

TOTAL DES INDICATEURS

=====

=====

TOTAL: 13 variables

RÉPARTITION PESTEL

=====

=====

P – Politique	: 2 variables (political_stability, control_corruption)
E – Économique	: 4 variables (gdp_per_capita, gdp_growth, inflation, chicken_imports_tonnes)
S – Social	: 3 variables (population, urban_pop_pct, meat_consumption_kcal_day)
T – Technologique	: 2 variables (logistics_index, distance_from_france)
E – Environnemental	: 1 variable (co2_per_capita)
L – Légal	: 1 variable (tariff_rate)

LISTE COMPLÈTE ALPHABÉTIQUE

=====

=====

1. chicken_imports_tonnes
2. co2_per_capita
3. control_corruption
4. distance_from_france
5. gdp_growth
6. gdp_per_capita
7. inflation
8. logistics_index
9. meat_consumption_kcal_day
10. political_stability
11. population
12. tariff_rate
13. urban_pop_pct

```
In [26]: print("🔗 FUSION DE TOUS LES DATASETS")
print("="*80)

# ÉTAPE 1 : Fusionner WDI + WGI
print("\n1 Fusion WDI + WGI...")
wb_data = wdi_final.merge(
    wgi_final[['Country Code', 'political_stability', 'control_corruption']],
    on='Country Code',
    how='outer'
)
print(f"✅ Résultat: {wb_data.shape}")

# ÉTAPE 2 : Ajouter CEPII
print("\n2 Ajout CEPII (Distances)...")
wb_data = wb_data.merge(
    cepii_final,
    on='Country Code',
    how='left'
)
print(f"✅ Résultat: {wb_data.shape}")

# ÉTAPE 3 : Ajouter FAO Food Balance
print("\n3 Ajout FAO Food Balance...")
wb_data = wb_data.merge(
    fao_food_final[['Country Code', 'meat_consumption_kcal_day']],
    on='Country Code',
    how='left'
)
print(f"✅ Résultat: {wb_data.shape}")

# ÉTAPE 4 : Ajouter FAO Trade
print("\n4 Ajout FAO Trade...")
final_dataset = wb_data.merge(
    fao_trade_final[['Country Code', 'chicken_imports_tonnes']],
    on='Country Code',
    how='left'
)
print(f"✅ Résultat: {final_dataset.shape}")

print("\n" + "="*80)
print("🎉 DATASET FINAL CRÉÉ !")
print("="*80)

# Stats de complétude
print("\n📊 ANALYSE DE COMPLÉTUDE")
```

```
print("="*80)

print(f"\nNombre total de pays: {len(final_dataset)}")

# NaN par variable
print("\nNaN par variable:")
for col in final_dataset.columns[2:]: # Skip Country Name et Code
    nan_count = final_dataset[col].isna().sum()
    valid = len(final_dataset) - nan_count
    pct = (nan_count / len(final_dataset)) * 100
    print(f"{col:30s}: {valid:3d} / {len(final_dataset):3d} ({pct:5.1f}%")

# Pays avec données complètes
complete = final_dataset.dropna()
print(f"\n✅ Pays avec TOUTES les 13 variables: {len(complete)}")

# Pays avec au moins 10/13 variables
threshold = 3 # max 3 NaN acceptés
semi_complete = final_dataset[final_dataset.isnull().sum(axis=1) <= thres
print(f"✅ Pays avec au moins 10/13 variables: {len(semi_complete)}")

print("\n📄 Aperçu (5 premiers pays):")
print(final_dataset.head())
```

FUSION DE TOUS LES DATASETS

=====

=====

- 1 Fusion WDI + WGI...
☒ Résultat: (275, 12)
- 2 Ajout CEPII (Distances)...
☒ Résultat: (275, 13)
- 3 Ajout FAO Food Balance...
☒ Résultat: (280, 14)
- 4 Ajout FAO Trade...
☒ Résultat: (280, 15)

=====

=====

 DATASET FINAL CRÉÉ !

=====

=====

ANALYSE DE COMPLÉTUDE

=====

=====

Nombre total de pays: 280

NaN par variable:

co2_per_capita	: 256 / 280 (8.6% NaN)
inflation	: 227 / 280 (18.9% NaN)
logistics_index	: 0 / 280 (100.0% NaN)
gdp_growth	: 256 / 280 (8.6% NaN)
gdp_per_capita	: 256 / 280 (8.6% NaN)
population	: 270 / 280 (3.6% NaN)
urban_pop_pct	: 270 / 280 (3.6% NaN)
tariff_rate	: 0 / 280 (100.0% NaN)
political_stability	: 216 / 280 (22.9% NaN)
control_corruption	: 217 / 280 (22.5% NaN)
distance_from_france	: 211 / 280 (24.6% NaN)
meat_consumption_kcal_day	: 179 / 280 (36.1% NaN)
chicken_imports_tonnes	: 149 / 280 (46.8% NaN)

- ☒ Pays avec TOUTES les 13 variables: 0
- ☒ Pays avec au moins 10/13 variables: 171

 Aperçu (5 premiers pays):

	Country Name	Country Code	co2_per_capita	inflation \
0	Aruba	ABW	5.002841	NaN
1	Africa Eastern and Southern	AFE	0.838104	7.399186
2	Afghanistan	AFG	0.280995	-4.644709
3	Africa Western and Central	AFW	0.494008	5.221168
4	Angola	AGO	0.743839	13.644102

	logistics_index	gdp_growth	gdp_per_capita	population	urban_pop_pct
0	NaN	7.706798	35718.753119	107359.0	61.835149
1	NaN	1.931160	1571.449189	750491370.0	37.772301
2	NaN	2.266944	413.757895	41454761.0	25.473053
3	NaN	3.662428	1841.855064	509398589.0	52.282908

	tariff_rate	political_stability	control_corruption	distance_from_france
0	NaN	1.432525	0.709075	7716.
1	NaN	NaN	NaN	
2	NaN	-2.484081	-1.154932	5500.
3	NaN	NaN	NaN	
4	NaN	-0.341505	-0.609920	6384.

	meat_consumption_kcal_day	chicken_imports_tonnes
0	NaN	NaN
1	NaN	NaN
2	32.48	NaN
3	NaN	NaN
4	92.63	236746.25

```
In [27]: print("🇫🇷 ANALYSE DE COMPLÉTUDE PAR ANNÉE (2019-2023)")
print("="*80)

# Préparer les données WDI/WGI par année
years_to_test = ['2019 [YR2019]', '2020 [YR2020]', '2021 [YR2021]', '2022 [YR2022]']
year_labels = [2019, 2020, 2021, 2022, 2023]

results = []

for year_col, year_label in zip(years_to_test, year_labels):
    print(f"\n{'='*80}")
    print(f"🔍 ANNÉE {year_label}")
    print(f"{'='*80}")

    # 1. WDI pour cette année
    wdi_year = wdi_clean[['Country Name', 'Country Code', 'Series Code',
                        'meat_consumption_kcal_day', 'chicken_imports_tonnes']]
    wdi_pivot = wdi_year.pivot(
        index=['Country Name', 'Country Code'],
        columns='Series Code',
        values=year_col
    ).reset_index().rename(columns=rename_wdi)

    # 2. WGI pour cette année - CORRECTION ICI
    wgi_year = wgi_clean[['Country Name', 'Country Code', 'Series Code',
                        'political_stability', 'control_corruption']]
    wgi_pivot = wgi_year.pivot(
        index=['Country Name', 'Country Code'],
        columns='Series Code',
        values=year_col
    ).reset_index().rename(columns=rename_wgi)

    # 3. Fusionner WDI + WGI
    wb_year = wdi_pivot.merge(
        wgi_pivot[['Country Code', 'political_stability', 'control_corruption',
                    'political_stability', 'control_corruption']],
        on='Country Code',
        how='outer'
    )

    # 4. Ajouter CEPII (pas de dimension temporelle)
```



```

wb_year = wb_year.merge(cepii_final, on='Country Code', how='left')

# 5. FAO Food Balance pour cette année
fao_food_year = food_balance[
    (food_balance['Item'] == 'Meat') &
    (food_balance['Element'] == 'Food supply (kcal/capita/day)') &
    (food_balance['Year'] == year_label)
].copy()
fao_food_year['Value'] = pd.to_numeric(fao_food_year['Value'], errors
fao_food_year_clean = fao_food_year[['Area', 'Value']].copy()
fao_food_year_clean.columns = ['Area', 'meat_consumption_kcal_day']
fao_food_year_clean['Country Code'] = fao_food_year_clean['Area'].app
fao_food_year_clean = fao_food_year_clean[fao_food_year_clean['Countr

# 6. FAO Trade pour cette année
fao_trade_year = trade_detailed[
    (trade_detailed['Item'] == 'Meat of chickens, fresh or chilled')
    (trade_detailed['Element'] == 'Import quantity') &
    (trade_detailed['Year'] == year_label)
].copy()
fao_trade_year['Value'] = pd.to_numeric(fao_trade_year['Value'], erro
fao_trade_agg_year = fao_trade_year.groupby('Reporter Countries')['Va
fao_trade_agg_year.columns = ['Area', 'chicken_imports_tonnes']
fao_trade_agg_year['Country Code'] = fao_trade_agg_year['Area'].apply
fao_trade_agg_year = fao_trade_agg_year[fao_trade_agg_year['Country C

# 7. Fusionner tout
dataset_year = wb_year.merge(
    fao_food_year_clean[['Country Code', 'meat_consumption_kcal_day']]
    on='Country Code',
    how='left'
).merge(
    fao_trade_agg_year[['Country Code', 'chicken_imports_tonnes']],
    on='Country Code',
    how='left'
)

# Analyser complétude
total_countries = len(dataset_year)
complete_countries = len(dataset_year.dropna())

# NaN par variable
nan_summary = {}
for col in dataset_year.columns[2:]:
    nan_count = dataset_year[col].isna().sum()
    nan_pct = (nan_count / total_countries) * 100
    nan_summary[col] = nan_pct

# Moyennes de NaN
avg_nan = sum(nan_summary.values()) / len(nan_summary)

# Sauvegarder résultats
results.append({
    'year': year_label,
    'total_countries': total_countries,
    'complete_countries': complete_countries,
    'avg_nan_pct': avg_nan,
    'nan_summary': nan_summary
})

```

```

print(f"Pays total: {total_countries}")
print(f"Pays complets (0 NaN): {complete_countries}")
print(f"Moyenne NaN: {avg_nan:.1f}%")

# Top 3 variables avec plus de NaN
top_nan = sorted(nan_summary.items(), key=lambda x: x[1], reverse=True)
print(f"\nTop 3 variables avec plus de NaN:")
for var, pct in top_nan:
    print(f"    - {var}: {pct:.1f}%")

# Résumé comparatif
print("\n" + "="*80)
print("📊 COMPARAISON PAR ANNÉE")
print("="*80)
print(f"\n{'Année':<10} {'Pays total':<15} {'Pays complets':<15} {'Moyenne NaN':<15}")
print("-"*80)
for r in results:
    print(f"{r['year']:<10} {r['total_countries']:<15} {r['complete_countries']:<15} {r['avg_nan_pct']:<15}")

# Meilleure année
best_year = max(results, key=lambda x: x['complete_countries'])
print(f"\n🏆 MEILLEURE ANNÉE: {best_year['year']}")
print(f"    → {best_year['complete_countries']} pays avec données complètes")
print(f"    → {best_year['avg_nan_pct']:.1f}% de NaN en moyenne")

```

 ANALYSE DE COMPLÉTUDE PAR ANNÉE (2019–2023) ANNÉE 2019

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.4%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- tariff_rate: 49.6%
- chicken_imports_tonnes: 43.9%

 ANNÉE 2020

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.0%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- chicken_imports_tonnes: 43.9%
- tariff_rate: 42.1%

 ANNÉE 2021

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.0%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- chicken_imports_tonnes: 44.6%
- tariff_rate: 41.8%

 ANNÉE 2022

Pays total: 280
Pays complets (0 NaN): 94
Moyenne NaN: 21.3%

Top 3 variables avec plus de NaN:

- tariff_rate: 47.5%
- chicken_imports_tonnes: 45.0%
- meat_consumption_kcal_day: 36.1%

ANNÉE 2023

Pays total: 280
 Pays complets (0 NaN): 0
 Moyenne NaN: 31.1%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- tariff_rate: 100.0%
- chicken_imports_tonnes: 46.8%

COMPARAISON PAR ANNÉE

Année	Pays total	Pays complets	Moyenne NaN %
2019	280	0	26.4
2020	280	0	26.0
2021	280	0	26.0
2022	280	94	21.3
2023	280	0	31.1

🏆 MEILLEURE ANNÉE: 2022
 → 94 pays avec données complètes
 → 21.3% de NaN en moyenne

```
In [28]: print("📊 ANALYSE DE COMPLÉTUDE PAR ANNÉE (2019-2023)")
print("="*80)

# Préparer les données WDI/WGI par année
years_to_test = ['2019 [YR2019]', '2020 [YR2020]', '2021 [YR2021]', '2022 [YR2022]']
year_labels = [2019, 2020, 2021, 2022, 2023]

results = []

for year_col, year_label in zip(years_to_test, year_labels):
    print(f"\n{'='*80}")
    print(f"🔍 ANNÉE {year_label}")
    print(f"{'='*80}")

    # 1. WDI pour cette année
    wdi_year = wdi_clean[['Country Name', 'Country Code', 'Series Code',
                        'Year Label']]
    wdi_pivot = wdi_year.pivot(
        index=['Country Name', 'Country Code'],
        columns='Series Code',
        values=year_col
    ).reset_index().rename(columns=rename_dict)

    # 2. WGI pour cette année - CORRECTION ICI
    wgi_year = wgi_clean[['Country Name', 'Country Code', 'Series Code',
                        'Year Label']]
    wgi_pivot = wgi_year.pivot(
        index=['Country Name', 'Country Code'],
        columns='Series Code',
```

```

        values=year_col
    ).reset_index().rename(columns=rename_wgi)

# 3. Fusionner WDI + WGI
wb_year = wdi_pivot.merge(
    wgi_pivot[['Country Code', 'political_stability', 'control_corruption'],
              on='Country Code',
              how='outer'
)

# 4. Ajouter CEPII (pas de dimension temporelle)
wb_year = wb_year.merge(cepii_final, on='Country Code', how='left')

# 5. FAO Food Balance pour cette année
fao_food_year = food_balance[
    (food_balance['Item'] == 'Meat') &
    (food_balance['Element'] == 'Food supply (kcal/capita/day)') &
    (food_balance['Year'] == year_label)
].copy()
fao_food_year['Value'] = pd.to_numeric(fao_food_year['Value'], errors='coerce')
fao_food_year_clean = fao_food_year[['Area', 'Value']].copy()
fao_food_year_clean.columns = ['Area', 'meat_consumption_kcal_day']
fao_food_year_clean['Country Code'] = fao_food_year_clean['Area'].apply(lambda x: x.split('_')[0])
fao_food_year_clean = fao_food_year_clean[fao_food_year_clean['Country Code'].notna()]

# 6. FAO Trade pour cette année
fao_trade_year = trade_detailed[
    (trade_detailed['Item'] == 'Meat of chickens, fresh or chilled') &
    (trade_detailed['Element'] == 'Import quantity') &
    (trade_detailed['Year'] == year_label)
].copy()
fao_trade_year['Value'] = pd.to_numeric(fao_trade_year['Value'], errors='coerce')
fao_trade_agg_year = fao_trade_year.groupby('Reporter Countries')['Value'].sum()
fao_trade_agg_year.columns = ['Area', 'chicken_imports_tonnes']
fao_trade_agg_year['Country Code'] = fao_trade_agg_year['Area'].apply(lambda x: x.split('_')[0])
fao_trade_agg_year = fao_trade_agg_year[fao_trade_agg_year['Country Code'].notna()]

# 7. Fusionner tout
dataset_year = wb_year.merge(
    fao_food_year_clean[['Country Code', 'meat_consumption_kcal_day']],
    on='Country Code',
    how='left'
).merge(
    fao_trade_agg_year[['Country Code', 'chicken_imports_tonnes']],
    on='Country Code',
    how='left'
)

# Analyser complétude
total_countries = len(dataset_year)
complete_countries = len(dataset_year.dropna())

# NaN par variable
nan_summary = {}
for col in dataset_year.columns[2:]:
    nan_count = dataset_year[col].isna().sum()
    nan_pct = (nan_count / total_countries) * 100
    nan_summary[col] = nan_pct

# Moyennes de NaN

```

```

avg_nan = sum(nan_summary.values()) / len(nan_summary)

# Sauvegarder résultats
results.append({
    'year': year_label,
    'total_countries': total_countries,
    'complete_countries': complete_countries,
    'avg_nan_pct': avg_nan,
    'nan_summary': nan_summary
})

print(f"Pays total: {total_countries}")
print(f"Pays complets (0 NaN): {complete_countries}")
print(f"Moyenne NaN: {avg_nan:.1f}%")

# Top 3 variables avec plus de NaN
top_nan = sorted(nan_summary.items(), key=lambda x: x[1], reverse=True)
print(f"\nTop 3 variables avec plus de NaN:")
for var, pct in top_nan:
    print(f" - {var}: {pct:.1f}%")

# Résumé comparatif
print("\n" + "="*80)
print("📊 COMPARAISON PAR ANNÉE")
print("="*80)
print(f"\n{'Année':<10} {'Pays total':<15} {'Pays complets':<15} {'Moyenn")
print("-"*80)
for r in results:
    print(f"{r['year']:<10} {r['total_countries']:<15} {r['complete_count")

# Meilleure année
best_year = max(results, key=lambda x: x['complete_countries'])
print(f"\n🏆 MEILLEURE ANNÉE: {best_year['year']}")
print(f" → {best_year['complete_countries']} pays avec données complète")
print(f" → {best_year['avg_nan_pct']:.1f}% de NaN en moyenne")

```

 ANALYSE DE COMPLÉTUDE PAR ANNÉE (2019–2023) ANNÉE 2019

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.4%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- tariff_rate: 49.6%
- chicken_imports_tonnes: 43.9%

 ANNÉE 2020

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.0%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- chicken_imports_tonnes: 43.9%
- tariff_rate: 42.1%

 ANNÉE 2021

Pays total: 280
Pays complets (0 NaN): 0
Moyenne NaN: 26.0%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- chicken_imports_tonnes: 44.6%
- tariff_rate: 41.8%

 ANNÉE 2022

Pays total: 280
Pays complets (0 NaN): 94
Moyenne NaN: 21.3%

Top 3 variables avec plus de NaN:

- tariff_rate: 47.5%
- chicken_imports_tonnes: 45.0%
- meat_consumption_kcal_day: 36.1%

ANNÉE 2023

Pays total: 280

Pays complets (0 NaN): 0

Moyenne NaN: 31.1%

Top 3 variables avec plus de NaN:

- logistics_index: 100.0%
- tariff_rate: 100.0%
- chicken_imports_tonnes: 46.8%

COMPARAISON PAR ANNÉE

Année	Pays total	Pays complets	Moyenne NaN %
2019	280	0	26.4
2020	280	0	26.0
2021	280	0	26.0
2022	280	94	21.3
2023	280	0	31.1

🏆 MEILLEURE ANNÉE: 2022

- 94 pays avec données complètes
- 21.3% de NaN en moyenne

```
In [29]: # Voir les indicateurs WDI disponibles mais non utilisés
print("📊 INDICATEURS WDI DISPONIBLES (non utilisés)")
print("="*80)

indicateurs_wdi_dispo = wdi['Series Name'].unique()
indicateurs_wdi_utilises = [
    'GDP per capita (current US$)',
    'GDP growth (annual %)',
    'Inflation, consumer prices (annual %)',
    'Population, total',
    'Urban population (% of total population)',
    'Logistics performance index: Overall (1=low to 5=high)',
    'Carbon dioxide (CO2) emissions excluding LULUCF per capita (t CO2e/capita)',
    'Tariff rate, applied, weighted mean, all products (%)'
]

indicateurs_non_utilises = [ind for ind in indicateurs_wdi_dispo if ind not in indicateurs_wdi_utilises]

print("\nIndicateurs disponibles pour remplacer tariff_rate:")
for idx, ind in enumerate(indicateurs_non_utilises, 1):
    print(f"{idx:2d}. {ind}")
```


INDICATEURS WDI DISPONIBLES (non utilisés)

Indicateurs disponibles pour remplacer tariff_rate:

1. GNI per capita, Atlas method (current US\$)
2. Trade (% of GDP)
3. Foreign direct investment, net inflows (BoP, current US\$)
4. Exports of goods and services (% of GDP)
5. Life expectancy at birth, total (years)
6. Literacy rate, adult total (% of people ages 15 and above)
7. Gini index
8. Population growth (annual %)
9. Access to electricity (% of population)
10. Individuals using the Internet (% of population)
11. Agricultural land (% of land area)
12. Electric power consumption (kWh per capita)
13. Current health expenditure per capita (current US\$)

```
In [30]: print("🔍 TEST DE DISPONIBILITÉ – Année 2022")
print("="*80)

# Liste des candidats à tester
candidats = [
    'Trade (% of GDP)',
    'Individuals using the Internet (% of population)',
    'Exports of goods and services (% of GDP)',
    'Access to electricity (% of population)',
    'Agricultural land (% of land area)',
    'Population growth (annual %)',
    'Life expectancy at birth, total (years)',
    'Foreign direct investment, net inflows (BoP, current US$)',
]

resultats = []

for candidat in candidats:
    # Extraire données 2022
    data_2022 = wdi[wdi['Series Name'] == candidat][['Country Code', '2022']]

    # Convertir en numérique
    data_2022['2022 [YR2022]'] = data_2022['2022 [YR2022]'].replace('.', '')
    data_2022['2022 [YR2022]'] = pd.to_numeric(data_2022['2022 [YR2022]'])

    # Calculer NaN
    nan_count = data_2022['2022 [YR2022]'].isna().sum()
    total = len(data_2022)
    valid = total - nan_count
    nan_pct = (nan_count / total) * 100

    resultats.append({
        'indicateur': candidat,
        'valid': valid,
        'nan_count': nan_count,
        'nan_pct': nan_pct
    })

print(f"\n{candidat[:50]:50s}")
print(f"    Valeurs: {valid:3d} / {total:3d} ({nan_pct:5.1f}% NaN)")
```

```

# Trier par meilleure disponibilité
resultats_sorted = sorted(resultats, key=lambda x: x['nan_pct'])

print("\n" + "="*80)
print("🏆 CLASSEMENT PAR DISPONIBILITÉ (du meilleur au pire)")
print("="*80)

for idx, r in enumerate(resultats_sorted, 1):
    print(f"{idx}. {r['indicateur'][:50]:50s} → {r['nan_pct']:5.1f}% NaN")

print("\n" + "="*80)
print("💡 RECOMMANDATION")
print("="*80)

best = resultats_sorted[0]
print(f"\n👑 MEILLEUR CHOIX: {best['indicateur']}")
print(f"    → {best['valid']} pays avec données")
print(f"    → Seulement {best['nan_pct']:.1f}% de NaN")

# Dimension PESTEL
if 'Trade' in best['indicateur'] or 'Export' in best['indicateur'] or 'FD':
    dimension = "Économique/Légal (remplace bien tariff_rate)"
elif 'Internet' in best['indicateur'] or 'electricity' in best['indicateur']:
    dimension = "Technologique"
elif 'Agricultural' in best['indicateur']:
    dimension = "Environnemental/Économique"
else:
    dimension = "Social"

print(f"    → Dimension PESTEL: {dimension}")

```

🔍 TEST DE DISPONIBILITÉ – Année 2022

Trade (% of GDP)

Valeurs: 224 / 266 (15.8% NaN)

Individuals using the Internet (% of population)

Valeurs: 215 / 266 (19.2% NaN)

Exports of goods and services (% of GDP)

Valeurs: 226 / 266 (15.0% NaN)

Access to electricity (% of population)

Valeurs: 263 / 266 (1.1% NaN)

Agricultural land (% of land area)

Valeurs: 257 / 266 (3.4% NaN)

Population growth (annual %)

Valeurs: 264 / 266 (0.8% NaN)

Life expectancy at birth, total (years)

Valeurs: 265 / 266 (0.4% NaN)

Foreign direct investment, net inflows (BoP, curre

Valeurs: 250 / 266 (6.0% NaN)

🏆 CLASSEMENT PAR DISPONIBILITÉ (du meilleur au pire)

1. Life expectancy at birth, total (years)	→	0.4% NaN (265 pa ys)
2. Population growth (annual %)	→	0.8% NaN (264 pa ys)
3. Access to electricity (% of population)	→	1.1% NaN (263 pa ys)
4. Agricultural land (% of land area)	→	3.4% NaN (257 pa ys)
5. Foreign direct investment, net inflows (BoP, curre	→	6.0% NaN (250 pa ys)
6. Exports of goods and services (% of GDP)	→	15.0% NaN (226 pa ys)
7. Trade (% of GDP)	→	15.8% NaN (224 pa ys)
8. Individuals using the Internet (% of population)	→	19.2% NaN (215 pa ys)

💡 RECOMMANDATION

🏆 MEILLEUR CHOIX: Life expectancy at birth, total (years)

→ 265 pays avec données

→ Seulement 0.4% de NaN

→ Dimension PESTEL: Social

```

In [31]: print("\n CRÉATION DATASET FINAL - 2 ANNÉES (2021 + 2022)")
print("="*80)

# =====
# FONCTION POUR CRÉER UN DATASET POUR UNE ANNÉE DONNÉE
# =====

def create_dataset_for_year(year_col, year_label):
    """Créer dataset complet pour une année donnée"""

    # 1. WDI (sans tariff_rate)
    indicateurs_wdi_finaux = [
        'GDP per capita (current US$)',
        'GDP growth (annual %)',
        'Inflation, consumer prices (annual %)',
        'Population, total',
        'Urban population (% of total population)',
        'Logistics performance index: Overall (1=low to 5=high)',
        'Carbon dioxide (CO2) emissions excluding LULUCF per capita (t CO
    ]

    wdi_year = wdi[wdi['Series Name'].isin(indicateurs_wdi_finaux)].copy()

    # Convertir en numérique
    wdi_year[year_col] = wdi_year[year_col].replace('..', np.nan)
    wdi_year[year_col] = pd.to_numeric(wdi_year[year_col], errors='coerce')

    # Pivoter
    wdi_pivot = wdi_year[['Country Code', 'Series Code', year_col]].pivot(
        index='Country Code',
        columns='Series Code',
        values=year_col
    ).reset_index()

    # Renommer
    rename_dict_final = {
        'NY.GDP.PCAP.CD': 'gdp_per_capita',
        'NY.GDP.MKTP.KD.ZG': 'gdp_growth',
        'FP.CPI.TOTL.ZG': 'inflation',
        'SP.POP.TOTL': 'population',
        'SP.URB.TOTL.IN.ZS': 'urban_pop_pct',
        'LP.LPI.OVRL.XQ': 'logistics_index',
        'EN.GHG.CO2.PC.CE.AR5': 'co2_per_capita',
    }
    wdi_pivot = wdi_pivot.rename(columns=rename_dict_final)

    # 2. WGI
    wgi_year = wgi[wgi['Series Name'].isin([
        'Political Stability and Absence of Violence/Terrorism: Estimate',
        'Control of Corruption: Estimate'
    ])].copy()

    wgi_year[year_col] = wgi_year[year_col].replace('..', np.nan)
    wgi_year[year_col] = pd.to_numeric(wgi_year[year_col], errors='coerce')

    wgi_pivot = wgi_year[['Country Code', 'Series Code', year_col]].pivot(
        index='Country Code',
        columns='Series Code',
        values=year_col
    )

```

```

).reset_index()

rename_wgi_final = {
    'PV.EST': 'political_stability',
    'CC.EST': 'control_corruption'
}
wgi_pivot = wgi_pivot.rename(columns=rename_wgi_final)

# 3. Fusionner WDI + WGI
wb_year = wdi_pivot.merge(
    wgi_pivot[['Country Code', 'political_stability', 'control_corruption'],
    on='Country Code',
    how='outer'
)

# 4. Ajouter CEPII
wb_year = wb_year.merge(cepii_final, on='Country Code', how='left')

# 5. FAO Food Balance
fao_food_year = food_balance[
    (food_balance['Item'] == 'Meat') &
    (food_balance['Element'] == 'Food supply (kcal/capita/day)') &
    (food_balance['Year'] == year_label)
].copy()
fao_food_year['Value'] = pd.to_numeric(fao_food_year['Value'], errors='coerce')
fao_food_year_clean = fao_food_year[['Area', 'Value']].copy()
fao_food_year_clean.columns = ['Area', 'meat_consumption_kcal_day']
fao_food_year_clean['Country Code'] = fao_food_year_clean['Area'].apply(lambda x: x.split('_')[0])
fao_food_year_clean = fao_food_year_clean[fao_food_year_clean['Country Code'] != '']

# 6. FAO Trade
fao_trade_year = trade_detailed[
    (trade_detailed['Item'] == 'Meat of chickens, fresh or chilled') &
    (trade_detailed['Element'] == 'Import quantity') &
    (trade_detailed['Year'] == year_label)
].copy()
fao_trade_year['Value'] = pd.to_numeric(fao_trade_year['Value'], errors='coerce')
fao_trade_agg_year = fao_trade_year.groupby('Reporter Countries')['Value'].sum()
fao_trade_agg_year.columns = ['Area', 'chicken_imports_tonnes']
fao_trade_agg_year['Country Code'] = fao_trade_agg_year['Area'].apply(lambda x: x.split('_')[0])
fao_trade_agg_year = fao_trade_agg_year[fao_trade_agg_year['Country Code'] != '']

# 7. Fusionner tout
dataset = wb_year.merge(
    fao_food_year_clean[['Country Code', 'meat_consumption_kcal_day']],
    on='Country Code',
    how='left'
).merge(
    fao_trade_agg_year[['Country Code', 'chicken_imports_tonnes']],
    on='Country Code',
    how='left'
)

return dataset

# =====
# CRÉER LES DATASETS POUR 2021 ET 2022
# =====

print("\n🇫🇷 Création dataset 2021...")

```

```

dataset_2021 = create_dataset_for_year('2021 [YR2021]', 2021)
print(f"✅ Dataset 2021: {dataset_2021.shape}")

print("\n📊 Création dataset 2022...")
dataset_2022 = create_dataset_for_year('2022 [YR2022]', 2022)
print(f"✅ Dataset 2022: {dataset_2022.shape}")

# =====
# RENOMMER LES COLONNES AVEC SUFFIXES _2021 et _2022
# =====

print("\n🔄 Ajout des suffixes d'année...")

# Renommer 2021
cols_2021 = {col: f"{col}_2021" for col in dataset_2021.columns if col !=
dataset_2021 = dataset_2021.rename(columns=cols_2021)

# Renommer 2022
cols_2022 = {col: f"{col}_2022" for col in dataset_2022.columns if col !=
dataset_2022 = dataset_2022.rename(columns=cols_2022)

# =====
# FUSIONNER LES 2 ANNÉES
# =====

print("\n🔗 Fusion des 2 années...")
final_dataset = dataset_2021.merge(dataset_2022, on='Country Code', how='

print(f"\n✅ Dataset final créé: {final_dataset.shape}")
print(f"Pays: {len(final_dataset)}")
print(f"Colonnes: {len(final_dataset.columns)}")

# =====
# ANALYSE FINALE
# =====

print("\n" + "="*80)
print("📊 ANALYSE FINALE")
print("="*80)

print("\n📋 Colonnes:")
print(final_dataset.columns.tolist())

print("\n📊 Aperçu (5 premières lignes):")
print(final_dataset.head())

# Compter pays avec données complètes
complete = final_dataset.dropna()
print(f"\n✅ Pays avec TOUTES les données (0 NaN): {len(complete)}")

# Pays avec au moins 20/24 variables
threshold = 4 # max 4 NaN
semi_complete = final_dataset[final_dataset.isnull().sum(axis=1) <= thres
print(f"✅ Pays avec au moins 20/24 variables: {len(semi_complete)}")

print("\n🌟 RÉSUMÉ:")
print(f" - 12 variables × 2 années = 24 colonnes de données")
print(f" - {len(final_dataset)} pays au total")
print(f" - {len(complete)} pays avec données 100% complètes")

```

🔧 CRÉATION DATASET FINAL – 2 ANNÉES (2021 + 2022)

📊 Création dataset 2021...

✅ Dataset 2021: (280, 13)

📊 Création dataset 2022...

✅ Dataset 2022: (280, 13)

🔄 Ajout des suffixes d'année...

🔗 Fusion des 2 années...

✅ Dataset final créé: (290, 25)

Pays: 290

Colonnes: 25

📊 ANALYSE FINALE

📁 Colonnes:

['Country Code', 'co2_per_capita_2021', 'inflation_2021', 'logistics_index_2021', 'gdp_growth_2021', 'gdp_per_capita_2021', 'population_2021', 'urban_pop_pct_2021', 'political_stability_2021', 'control_corruption_2021', 'distance_from_france_2021', 'meat_consumption_kcal_day_2021', 'chicken_imports_tonnes_2021', 'co2_per_capita_2022', 'inflation_2022', 'logistics_index_2022', 'gdp_growth_2022', 'gdp_per_capita_2022', 'population_2022', 'urban_pop_pct_2022', 'political_stability_2022', 'control_corruption_2022', 'distance_from_france_2022', 'meat_consumption_kcal_day_2022', 'chicken_imports_tonnes_2022']

📊 Aperçu (5 premières lignes):

	Country Code	co2_per_capita_2021	inflation_2021	logistics_index_2021
0	ABW	4.711235	NaN	NaN
1	AFE	0.899170	6.824727	NaN
2	AFG	0.312157	5.133203	NaN
3	AFW	0.526349	3.745568	NaN
4	AGO	0.740293	25.754295	NaN

	gdp_growth_2021	gdp_per_capita_2021	population_2021	urban_pop_pct_2021
0	14.730616	26749.329609	107700.0	61.9275
1	4.578772	1562.416175	713090928.0	36.9085
2	-20.738839	356.496214	40000412.0	25.3478
3	2.549691	2112.794076	485920997.0	51.0723
4	2.102753	2303.908127	34532429.0	68.2524

	political_stability_2021	control_corruption_2021	...
0	1.431727	0.800939	...
1	NaN	NaN	...

2	-2.518676	-1.152266	...
3	NaN	NaN	...
4	-0.710481	-0.649476	...

	logistics_index_2022	gdp_growth_2022	gdp_per_capita_2022 \
0	NaN	10.636431	30975.998912
1	2.618182	3.722717	1679.327622
2	1.900000	-6.240172	357.261153
3	2.473333	4.472795	2138.473153
4	2.100000	4.216003	3682.113151

	population_2022	urban_pop_pct_2022	political_stability_2022 \
0	107310.0	61.881166	1.483080
1	731821393.0	37.360578	NaN
2	40578842.0	25.393925	-2.544956
3	497387180.0	51.681694	NaN
4	35635029.0	69.050619	-0.630109

	control_corruption_2022	distance_from_france_2022 \
0	0.756487	7716.752
1	NaN	NaN
2	-1.183684	5500.665
3	NaN	NaN
4	-0.612376	6384.805

	meat_consumption_kcal_day_2022	chicken_imports_tonnes_2022
0	NaN	NaN
1	NaN	NaN
2	34.19	NaN
3	NaN	NaN
4	115.51	308122.7

[5 rows x 25 columns]

- ✓ Pays avec TOUTES les données (0 NaN): 0
- ✓ Pays avec au moins 20/24 variables: 181

🔗 RÉSUMÉ:

- 12 variables × 2 années = 24 colonnes de données
- 290 pays au total
- 0 pays avec données 100% complètes

```
In [32]: print("🔧 SUPPRESSION COMPLÈTE : logistics_index (2021 ET 2022)")
print("="*80)

# Supprimer logistics_index pour les 2 années
final_dataset = final_dataset.drop(columns=['logistics_index_2021', 'logistics_index_2022'])

print(f"✓ Après suppression: {final_dataset.shape}")
print(f"Colonnes: {len(final_dataset.columns) - 1} (Country Code + {len(final_dataset.columns) - 1} autres variables)")

# Nouvelle analyse
print("\n🇺🇦 NOUVELLE ANALYSE")
print("="*80)

# Pays avec données complètes
complete = final_dataset.dropna()
print(f"\n✓ Pays avec TOUTES les données (0 NaN): {len(complete)}")

# Pays avec au moins 18/22 variables
```



```

threshold = 4 # max 4 NaN
semi_complete = final_dataset[final_dataset.isnull().sum(axis=1) <= thres
print(f"✅ Pays avec au moins 18/22 variables: {len(semi_complete)}")

# NaN par variable (afficher seulement > 20% NaN)
print("\n📦 Variables avec > 20% de NaN:")
for col in final_dataset.columns[1:]:
    nan_count = final_dataset[col].isna().sum()
    valid = len(final_dataset) - nan_count
    pct = (nan_count / len(final_dataset)) * 100
    if pct > 20:
        print(f"{col:40s}: {pct:5.1f}% NaN ({valid} pays)")

print("\n" + "="*80)
print("🎯 RÉSULTAT FINAL:")
print("="*80)
print(f"    - Variables: 11 × 2 années = 22 colonnes")
print(f"    - Total pays: {len(final_dataset)}")
print(f"    - Pays 100% complets: {len(complete)}")
print(f"    - Pays avec ≥18/22 variables: {len(semi_complete)}")

if len(complete) >= 100:
    print(f"\n🎉 OBJECTIF ATTEINT ! {len(complete)} ≥ 100 pays !")
else:
    print(f"\n⚠️ On a {len(complete)} pays complets (objectif: 100)")
    print(f"    Mais {len(semi_complete)} pays avec ≥18/22 variables")

```

SUPPRESSION COMPLÈTE : logistics_index (2021 ET 2022)

=====

✓ Après suppression: (290, 23)
Colonnes: 22 (Country Code + 22 variables)

NOUVELLE ANALYSE

=====

=====

✓ Pays avec TOUTES les données (0 NaN): 134
✓ Pays avec au moins 18/22 variables: 197

Variables avec > 20% de NaN:

political_stability_2021	:	22.4% NaN (225 pays)
control_corruption_2021	:	23.1% NaN (223 pays)
distance_from_france_2021	:	23.8% NaN (221 pays)
meat_consumption_kcal_day_2021	:	34.8% NaN (189 pays)
chicken_imports_tonnes_2021	:	43.8% NaN (163 pays)
political_stability_2022	:	22.4% NaN (225 pays)
control_corruption_2022	:	22.4% NaN (225 pays)
distance_from_france_2022	:	23.8% NaN (221 pays)
meat_consumption_kcal_day_2022	:	34.8% NaN (189 pays)
chicken_imports_tonnes_2022	:	44.1% NaN (162 pays)

=====

=====

RÉSULTAT FINAL:

=====

=====

- Variables: 11 × 2 années = 22 colonnes
- Total pays: 290
- Pays 100% complets: 134
- Pays avec ≥18/22 variables: 197

 OBJECTIF ATTEINT ! 134 ≥ 100 pays !

Décision : suppression de `logistics_index`

L'analyse de complétude par année montre que 2022 est la meilleure année (94 pays complets) grâce à la présence de `logistics_index`. Cependant, cette variable reste absente pour 100% des pays en 2021 et 2023, ce qui introduirait une asymétrie temporelle dans le dataset bi-annuel (2021+2022).

Choix retenu : supprimer `logistics_index` des deux années pour maximiser la couverture pays (134 → objectif atteint) et garantir la cohérence temporelle entre 2021 et 2022.

```
In [33]: print("🗑 SUPPRESSION DES PAYS AVEC NaN")
print("="*80)

# Avant suppression
print(f"Avant : {len(final_dataset)} pays")

# Supprimer toutes les lignes avec au moins 1 NaN
final_dataset_clean = final_dataset.dropna()
```

```

# Après suppression
print(f"Après : {len(final_dataset_clean)} pays")
print(f"→ {len(final_dataset) - len(final_dataset_clean)} pays supprimés"

print("\n✅ DATASET FINAL 100% PROPRE")
print("="*80)
print(f"    - Pays : {len(final_dataset_clean)}")
print(f"    - Variables : 11 × 2 années = 22 colonnes")
print(f"    - NaN : 0 (aucun)")
print(f"    - Prêt pour ML : OUI ✅")

# Aperçu
print("\n📄 Aperçu du dataset propre:")
print(final_dataset_clean.head())

# Statistiques descriptives
print("\n📊 Statistiques descriptives (exemple – quelques variables):")
print(final_dataset_clean[['gdp_per_capita_2022', 'population_2022', 'inf

# Sauvegarder
print("\n💾 SAUVEGARDE...")
final_dataset_clean.to_csv('/Users/cordelia/Documents/Projet 11/New_noteb
print("✅ Fichier sauvegardé : /Users/cordelia/Documents/Projet 11/New_no

print("\n🎉 DATASET PRÊT POUR ACP + CLUSTERING !")

```

🗑 SUPPRESSION DES PAYS AVEC NaN

=====

=====

Avant : 290 pays
Après : 134 pays
→ 156 pays supprimés

✅ DATASET FINAL 100% PROPRE

=====

=====

- Pays : 134
- Variables : 11 × 2 années = 22 colonnes
- NaN : 0 (aucun)
- Prêt pour ML : OUI ✅

📄 Aperçu du dataset propre:

	Country Code	co2_per_capita_2021	inflation_2021	gdp_growth_2021	\
4	AGO	0.740293	25.754295	2.102753	
10	ARE	20.826928	0.179935	4.552786	
11	ARG	4.097697	48.409379	10.441812	
12	ARM	2.519461	7.184836	5.800000	
14	ATG	3.589644	2.062996	8.172863	

	gdp_per_capita_2021	population_2021	urban_pop_pct_2021	\
4	2303.908127	34532429.0	68.252416	
10	44118.504719	9575152.0	85.357191	
11	10738.017922	45312281.0	92.029278	
12	4685.179971	2962300.0	65.357288	
14	17348.600699	92349.0	24.574984	

	political_stability_2021	control_corruption_2021	\
4	-0.710481	-0.649476	
10	0.597572	1.150395	
11	0.000530	-0.422597	
12	-0.804041	0.048685	
14	0.949670	0.231869	

	distance_from_france_2021	...	inflation_2022	gdp_growth_2022	\
4	6384.805	...	21.355290	4.216003	
10	5180.430	...	5.291226	7.514484	
11	10932.340	...	72.430758	6.020745	
12	3402.570	...	8.640911	12.600000	
14	6739.046	...	7.531078	9.107806	

	gdp_per_capita_2022	population_2022	urban_pop_pct_2022	\
4	3682.113151	35635029.0	69.050619	
10	50759.758923	10074977.0	85.514735	
11	13962.189409	45407904.0	92.112838	
12	6571.974455	2969200.0	65.562446	
14	20105.198909	92840.0	24.511448	

	political_stability_2022	control_corruption_2022	\
4	-0.630109	-0.612376	
10	0.744183	1.155614	
11	-0.098594	-0.447222	
12	-0.740756	0.027907	
14	0.938082	0.310728	

	distance_from_france_2022	meat_consumption_kcal_day_2022	\
4	6384.805	115.51	

10	5180.430	274.74
11	10932.340	559.98
12	3402.570	289.81
14	6739.046	385.64

	chicken_imports_tonnes_2022
4	308122.70
10	615447.92
11	9792.75
12	37492.20
14	6407.00

[5 rows x 23 columns]

 Statistiques descriptives (exemple – quelques variables):

	gdp_per_capita_2022	population_2022	inflation_2022
count	134.000000	1.340000e+02	134.000000
mean	18665.002775	8.780823e+07	13.188242
std	24232.904574	2.691262e+08	19.514108
min	578.251664	9.284000e+04	-1.610680
25%	3484.370598	3.028100e+06	5.686104
50%	7713.094217	1.057953e+07	8.290618
75%	22747.135805	4.766776e+07	13.916097
max	123719.658916	1.425423e+09	171.205491

 SAUVEGARDE...

 Fichier sauvegardé : /Users/cordelia/Documents/Projet 11/New_notebook/dataset_final_clean_134pays.csv

 DATASET PRÊT POUR ACP + CLUSTERING !

```
In [37]: dupes = final_dataset_clean[final_dataset_clean.duplicated(subset='Country Code')]
print(f"Lignes dupliquées : {len(dupes)}")
print(dupes['Country Code'].sort_values())
```

Lignes dupliquées : 12

```
42    CHN
43    CHN
44    CHN
45    CHN
186   NGA
187   NGA
188   NGA
189   NGA
284   ZAF
285   ZAF
286   ZAF
287   ZAF
```

Name: Country Code, dtype: str

```
In [38]: # =====
# SUPPRESSION DES DOUBLONS SUR Country Name
# =====
print(f"Avant déduplication : {len(final_dataset_clean)} lignes")

final_dataset_clean = final_dataset_clean.drop_duplicates(subset='Country Code')
print(f"Après déduplication : {len(final_dataset_clean)} lignes")

# Sauvegarder le fichier final propre
```

```
final_dataset_clean.to_csv('/Users/cordelia/Documents/Projet 11/New_notebook/updated.html')
print("✅ Fichier mis à jour sans doublons")
```

Avant déduplication : 134 lignes

Après déduplication : 125 lignes

✅ Fichier mis à jour sans doublons

```
In [41]: code_to_name = wdi_final[['Country Code', 'Country Name']].drop_duplicates()
final_dataset_clean = final_dataset_clean.merge(code_to_name, on='Country Code')
cols = ['Country Code', 'Country Name'] + [c for c in final_dataset_clean.columns if c not in ['Country Code', 'Country Name']]
final_dataset_clean = final_dataset_clean[cols]
print(final_dataset_clean[['Country Code', 'Country Name']].head(10))
```

	Country Code	Country Name
0	AGO	Angola
1	ARE	United Arab Emirates
2	ARG	Argentina
3	ARM	Armenia
4	ATG	Antigua and Barbuda
5	AUS	Australia
6	AUT	Austria
7	AZE	Azerbaijan
8	BEL	Belgium
9	BFA	Burkina Faso

```
In [47]: final_dataset_clean = final_dataset_clean.drop(columns=[c for c in final_dataset_clean.columns if c not in ['Country Code', 'Country Name']])

final_dataset_clean = final_dataset_clean.merge(
    wdi_final[['Country Code', 'Country Name']].drop_duplicates(),
    on='Country Code',
    how='left'
)

# Réorganiser
cols = ['Country Code', 'Country Name'] + [c for c in final_dataset_clean.columns if c not in ['Country Code', 'Country Name']]
final_dataset_clean = final_dataset_clean[cols]

# Vérifier
print(final_dataset_clean.columns.tolist())
print(f"Shape : {final_dataset_clean.shape}")

# Sauvegarder
final_dataset_clean.to_csv('/Users/cordelia/Documents/Projet 11/New_notebook/updated.html')
print("✅ Sauvegardé")
```

```
['Country Code', 'Country Name', 'co2_per_capita_2021', 'inflation_2021', 'gdp_growth_2021', 'gdp_per_capita_2021', 'population_2021', 'urban_pop_pct_2021', 'political_stability_2021', 'control_corruption_2021', 'distance_from_france_2021', 'meat_consumption_kcal_day_2021', 'chicken_imports_tonnes_2021', 'co2_per_capita_2022', 'inflation_2022', 'gdp_growth_2022', 'gdp_per_capita_2022', 'population_2022', 'urban_pop_pct_2022', 'political_stability_2022', 'control_corruption_2022', 'distance_from_france_2022', 'meat_consumption_kcal_day_2022', 'chicken_imports_tonnes_2022']
Shape : (125, 24)
✅ Sauvegardé
```

In []: